

Modern Computer Architecture Concepts & Formulas

CSA222 · all 15 units · definitions, key points, formulas, memory tricks. No questions.

How each topic is laid out

Definition — the exam-ready English definition (write this in answers).

Hinglish: simple explanation, only for the harder topics.

- Key points — highlighted parts are the most asked.

Formula box — with one small example, every step shown.

Trick: a hook to remember it.

Yaad rakho: the 2-3 lines you must not forget.

How to study it: (1) read the definition aloud once; (2) read the Hinglish and key points until the idea makes sense; (3) cover the formula box and try to write it; (4) the night before, read only the “Yaad rakho” boxes and the last-hour cheat sheet at the end.

Definitions follow the class slides and labs, checked against M. Morris Mano (Digital Design), Harris & Harris (Digital Design and Computer Architecture), Patterson & Hennessy (Computer Organization and Design) and GeeksforGeeks. Units 13.1, 13.5, 14 and 15 had no class slides and are taken from these textbooks.

Unit 1 · Boolean Algebra & Gates

1.1 Course introduction & roadmap

Computer architecture is the study of how a computer is designed and organised — from logic gates up to the instruction set that software uses.

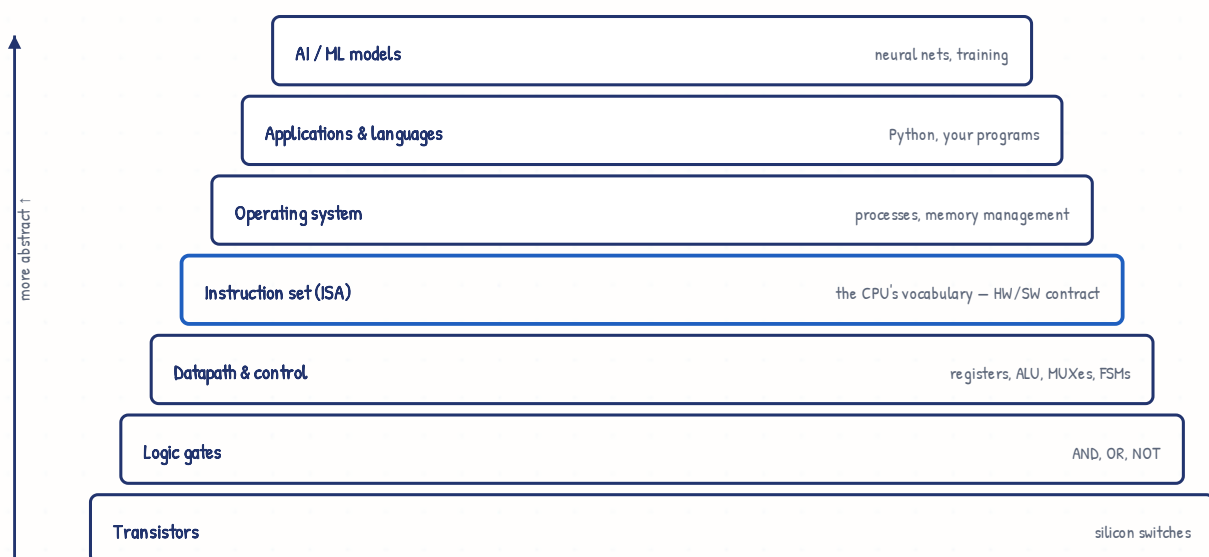
- **Architecture** = the “what”: what the programmer sees (instructions, registers, memory model).
- **Organisation** = the “how”: hidden implementation (how ADD is built, pipelines, caches, clock).
- Six course modules: digital foundations → sequential logic & memory → organisation & ISA → datapath & pipelining → memory hierarchy → parallelism.

Yaad rakho: Architecture = what (ISA). Organisation = how (implementation).

1.2 Why architecture matters — the abstraction stack

An **abstraction stack** is the set of layers through which a computer is built, each layer hiding the details of the one below it: applications & languages → operating system → **ISA** → datapath & control → logic gates → transistors.

- The **ISA (Instruction Set Architecture)** is the contract between software and hardware.
- Example: $A[i] = B[i] + C[i]$ in Python → MIPS `lw, lw, add, sw` → register file and ALU → XOR/AND gates → transistors.
- Chip flow (VLSI): spec → RTL (Verilog) → simulation → synthesis → netlist → place & route → verification → fabrication.



This course climbs the middle: logic gates → datapath & control → ISA

The abstraction stack

How to draw: Seven stacked bars, widest at the bottom (transistors); highlight the ISA bar and write “HW/SW contract”.

Yaad rakho: ISA sits between the OS and the datapath — the hardware/software contract.

1.3 Boolean logic basics – AND, OR, NOT, XOR (+ NAND, NOR, XNOR)

A **logic gate** is an electronic circuit that performs a Boolean function on one or more binary inputs (0 = LOW/false, 1 = HIGH/true) and produces a single binary output.

- **AND:** 1 only if ALL inputs are 1. **OR:** 1 if ANY input is 1. **NOT:** inverts.
- **NAND** = NOT of AND, **NOR** = NOT of OR (output 1 only when all inputs are 0).
- **XOR:** 1 when inputs are DIFFERENT; **XNOR:** 1 when inputs are EQUAL.
- NAND and NOR are **universal gates** (can build any circuit).

$$\begin{aligned} \text{AND } Y &= A \cdot B \quad | \quad \text{OR } Y = A + B \quad | \quad \text{NOT } Y = A' \quad | \quad \text{NAND } Y = (AB)' \quad | \quad \text{NOR } Y = (A + B)' \\ \text{XOR } Y &= A \oplus B = A'B + AB' \quad | \quad \text{XNOR } Y = (A \oplus B)' = A'B' + AB \end{aligned}$$



NOT
 $Y = A'$



AND
 $Y = A \cdot B$



OR
 $Y = A + B$



NAND
 $Y = (A \cdot B)'$



NOR
 $Y = (A + B)'$



XOR
 $Y = A \oplus B = A'B + AB'$



XNOR
 $Y = (A \oplus B)' = A'B' + AB$

Symbols of the seven gates with expressions

How to draw: AND = D-shape, OR = shield/curved back, XOR = OR with a second back curve; add a small bubble for NOT/NAND/NOR/XNOR.

Trick: XOR = "odd number of 1s $\rightarrow$ 1". $A = 1, B = 0$: only OR, XOR and NAND give 1.

Yaad rakho: XOR(1,1) = 0. NAND/NOR = AND/OR + bubble.

1.4 Truth tables

A **truth table** lists the output of a logic function for every possible combination of its inputs.

- n inputs $\rightarrow$ **2^n rows** (2 inputs $\rightarrow$ 4 rows, 3 $\rightarrow$ 8, 4 $\rightarrow$ 16).
- Write inputs in binary counting order: 000, 001, 010, ... so no row is missed.
- Truth table is the final check of any simplification.

Yaad rakho: rows = 2^n , always in binary counting order.

1.5 Boolean laws & simplification

Boolean algebra is the algebra of variables that take only the values 0 and 1, with the operations AND ($\cdot$), OR ($+$) and NOT ($'$); its laws let us rewrite an expression into a simpler equivalent one.

Identity $A + 0 = A$, $A \cdot 1 = A$ · Null $A + 1 = 1$, $A \cdot 0 = 0$ · Idempotent $A + A = A$, $A \cdot A = A$

Complement $A + A' = 1$, $A \cdot A' = 0$ · Involution $(A')' = A$

Distributive $A(B + C) = AB + AC$ and $A + BC = (A + B)(A + C)$

Absorption $A + AB = A$, $A(A + B) = A$ · $A + A'B = A + B$

De Morgan $(AB)' = A' + B'$, $(A + B)' = A'B'$ · Consensus $XY + X'Z + YZ = XY + X'Z$

Example — why $A + AB = A$: take A common $\rightarrow A(1 + B) \rightarrow 1 + B = 1 \rightarrow A \cdot 1 = A$.

Trick: De Morgan = "break the bar, change the sign" (+ becomes $\cdot$, $\cdot$ becomes +).

Yaad rakho: "+" is OR, so $A + 1 = 1$ (not 2). $A + A'B = A + B$ is the most used shortcut.

1.6 Worked simplification examples

Simplification means reducing a Boolean expression to fewer terms and literals (so fewer gates) while keeping the same truth table.

$AB + AB' = A(B + B') = A \cdot 1 = A$

$A(A + B) = AA + AB = A + AB = A$

$A'B'C + A'BC + AB'C + ABC + A'B \rightarrow$ first four all have C $\rightarrow C$ (all four AB combos = 1) = C $\rightarrow C + A'B$

Yaad rakho: look for a common factor, then use $X + X' = 1$ to kill the changing variable.

1.E1 Verilog & HDL basics (from the labs)

A **Hardware Description Language (HDL)** such as Verilog describes the structure and behaviour of digital circuits as text, so they can be **simulated** (behaviour checked) and **synthesised** (converted into real gates).

- module ... endmodule; ports input/output. wire is driven by assign; reg is assigned inside always/initial.
- Combinational: assign y = a & b; or always @(*) (with a default in case, else a latch is inferred).
- Sequential: always @(posedge clk) q <= d; — use non-blocking <=.
- Testbench: no ports, #10 delay, \$monitor, \$dumpfile/\$dumpvars, \$finish.

Yaad rakho: software runs one line at a time; hardware (HDL) runs all statements in parallel.

Unit 2 · Logic Minimisation & Universal Gates

2.1 Motivation for minimisation

Logic minimisation is the process of finding the simplest circuit (fewest gates and inputs) that implements a given Boolean function.

- Fewer gates → less area, less power, more speed, lower cost.
- Industry tools (Synopsys Design Compiler, Cadence Genus) do it automatically for billions of transistors.

Yaad rakho: 4 reasons — area, power, speed, cost.

2.2 Canonical forms: SOP and POS

A **minterm** is an AND (product) term containing every variable once, true for exactly one input row. A **maxterm** is an OR (sum) term containing every variable once, false for exactly one row. **SOP** (Sum of Products) = OR of the minterms where $F = 1$; **POS** (Product of Sums) = AND of the maxterms where $F = 0$.

Hinglish: SOP mein hum sirf un rows ko dekhte hain jahan output 1 hai, aur har row ka ek AND term banate hain. Rule: bit 1 hai toh variable seedha likho (A), bit 0 hai toh bar lagao (A'). POS mein ulta: sirf un rows ko dekho jahan output 0 hai, har row ka ek OR term banao, aur rule bhi ulta — bit 0 hai toh seedha (A), bit 1 hai toh bar (A'). Exam mein sabse zyada galti isi ulte rule mein hoti hai.

SOP rule: $1 \rightarrow X, 0 \rightarrow X'$. Row 101 $\rightarrow m_5 = AB'C$

POS rule: $0 \rightarrow X, 1 \rightarrow X'$. Row 011 $\rightarrow M_3 = (A + B' + C')$

$F = \sum m(1's \text{ rows}) = \prod M(0's \text{ rows})$; together they cover 0 to $2^n - 1$.

Example: $F = \sum m(1, 3, 5, 7) \rightarrow \prod M(0, 2, 4, 6)$. $SOP = A'B'C + A'BC + AB'C + ABC = A'C + AC = C$

Yaad rakho: SOP $\leftrightarrow$ 1s $\leftrightarrow$ minterm m. POS $\leftrightarrow$ 0s $\leftrightarrow$ maxterm M. The variable rules are opposite.

2.3 Karnaugh maps (2, 3, 4 variables)

A **Karnaugh map (K-map)** is a grid of the truth table in which adjacent cells differ in exactly one variable (Gray-code order), so that adjacent 1s can be grouped visually to obtain a minimal SOP expression.

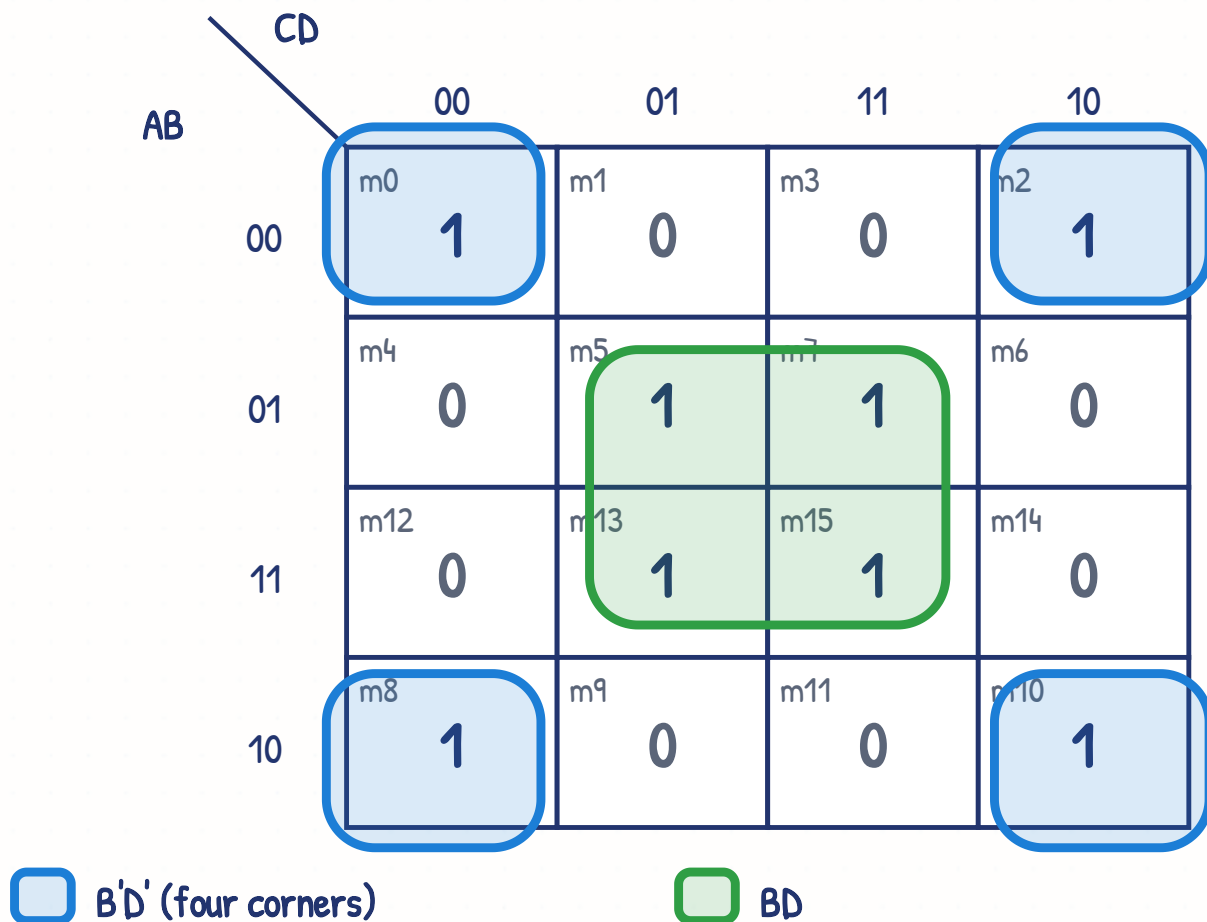
Hinglish: K-map basically truth table ko aise arrange karta hai ki pados wale boxes mein sirf ek hi variable badle. Isliye order 00, 01, 11, 10 hota hai (Gray code), 00, 01, 10, 11 nahi. Jab do padosi boxes mein 1 ho, toh jo variable badal raha hai woh cancel ho jaata hai ($B + B' = 1$). Isliye group jitna bada, expression utna chhota.

Groups sirf 1, 2, 4, 8, 16 size ke rectangle hote hain, diagonal nahi chalega, aur map ke kinare aapas mein jude hote hain — left-right, top-bottom aur chaaron corners ek group ban sakte hain.

- Order 00, 01, 11, 10 on both axes.
- Group sizes 1, 2, 4, 8, 16 only; rectangles; wrap-around and 4 corners allowed; overlapping allowed; diagonal not allowed.
- Make groups as BIG as possible and as FEW as possible.

A group of 2^k cells removes k variables — keep only the variables that stay constant inside the group.

Example: $\Sigma m(0, 2, 5, 7, 8, 10, 13, 15) \rightarrow$ corners 0, 2, 8, 10 have $B = 0, D = 0 \rightarrow B'D'$; centre 5, 7, 13, 15 have $B = 1, D = 1 \rightarrow BD \rightarrow F = B'D' + BD$



Wrap-around: $\Sigma m(0, 2, 5, 7, 8, 10, 13, 15) = B'D' + BD$

How to draw: Draw four quarter-loops at the corners and label them as one group.

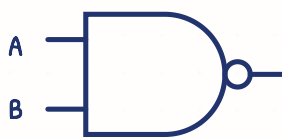
Trick: Gray code = "00, 01, 11, 10" — the 11 comes before 10.

Yaad rakho: four corners = one quad. Group of 4 kills 2 variables, group of 8 kills 3.

2.4 De Morgan's theorems & bubble pushing

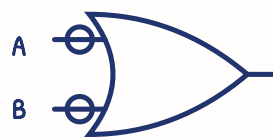
De Morgan's theorems state that the complement of a product equals the sum of the complements, $(AB)' = A' + B'$, and the complement of a sum equals the product of the complements, $(A + B)' = A'B'$. **Bubble pushing** is redrawing a gate using these laws by moving inversion bubbles from output to inputs.

- **NAND = OR with inverted inputs.** **NOR = AND with inverted inputs.**
- Two bubbles on one wire cancel each other.
- Nested: $\sim(AB + C) = (A' + B') \cdot C'$.



NAND: $(A \cdot B)'$

$\equiv$



OR of inverted inputs: $A' + B'$

Bubble pushing: **NAND $\equiv$ OR with inverted inputs**

How to draw: Draw an AND with an output bubble, then an OR with bubbles on both inputs, joined by $\equiv$.

Yaad rakho: break the bar, change the sign — AND $\leftrightarrow$ OR and every input flips.

2.5 Universal gates (NAND / NOR)

A **universal gate** is a gate from which every other logic gate (and so any Boolean function) can be built. NAND and NOR are universal.

Hinglish: NAND ke dono inputs jod do toh woh NOT ban jaata hai: $(A \cdot A)' = A'$. Ab NOT mil gaya, toh NAND ke baad NOT lagao $\rightarrow$ AND (2 gates). OR ke liye pehle A aur B dono ko ulta karo (2 NAND) aur phir NAND karo: $(A' \cdot B')' = A + B$ (De Morgan se) $\rightarrow$ 3 gates. NOR ke liye bilkul ulta hisaab: NOT 1, OR 2, AND 3. Factories NAND isliye pasand karti hain kyunki CMOS NAND sirf 4 transistors ka hota hai, AND 6 ka.

	NOT	AND	OR	XOR
NAND only	1	2	3	4
NOR only	1	3	2	5

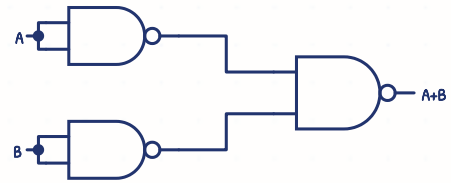
SOP (AND-OR) $\rightarrow$ **NAND-NAND**; POS (OR-AND) $\rightarrow$ **NOR-NOR**. $(A + B)(C + D)$ with 2-input NANDs needs 6.



NOT: $\text{NAND}(A,A)$



AND: 2 gates



OR: 3 gates

NOT, AND, OR from NAND gates

How to draw: Tie inputs together for NOT; NAND + NAND-inverter for AND; invert both inputs then NAND for OR.

Yaad rakho: NAND: 1-2-3 (NOT-AND-OR). NOR: 1-3-2. NAND = 4 transistors, AND = 6.

2.6 Don't-cares in K-maps

A **don't-care (X)** is an input combination whose output does not matter (because it never occurs or is never used); it may be taken as 1 or 0, whichever gives a simpler expression.

- Sources: BCD codes 10–15, unused states, impossible sensor combinations.
- Use an X only if it makes a group bigger; never make a group of only Xs.

Yaad rakho: X = "1 if it helps, 0 if it doesn't".

2.E1 Prime implicants, essential PIs, consensus

A **prime implicant (PI)** is a group of 1s that cannot be enlarged further. An **essential prime implicant (EPI)** is a PI that covers at least one 1 that no other PI covers.

- Minimal SOP = all EPIs + the fewest extra PIs to cover the remaining 1s.
- A cyclic map (e.g. $\Sigma m(0,1,2,5,6,7)$) has no EPI.
- Consensus: $XY + X'Z + YZ = XY + X'Z$ (YZ is redundant).

Yaad rakho: EPIs are always in the answer.

Unit 3 · Combinational Building Blocks

3.1 From gates to reusable blocks

A **combinational building block** is a standard circuit (MUX, DEMUX, decoder, adder) built from gates whose output depends only on its present inputs, reused again and again in larger designs.

- Why blocks: abstraction, reuse, easier verification, scale to millions of gates.
- In a CPU: MUX picks ALU operands and the next PC; decoder selects a register; adder does arithmetic and $PC + 4$.

3.2 Multiplexer (2:1, 4:1, cascading)

A **multiplexer (MUX)** is a combinational circuit that selects one of 2^n data inputs and passes it to a single output, according to the value on n select lines (a "data selector").

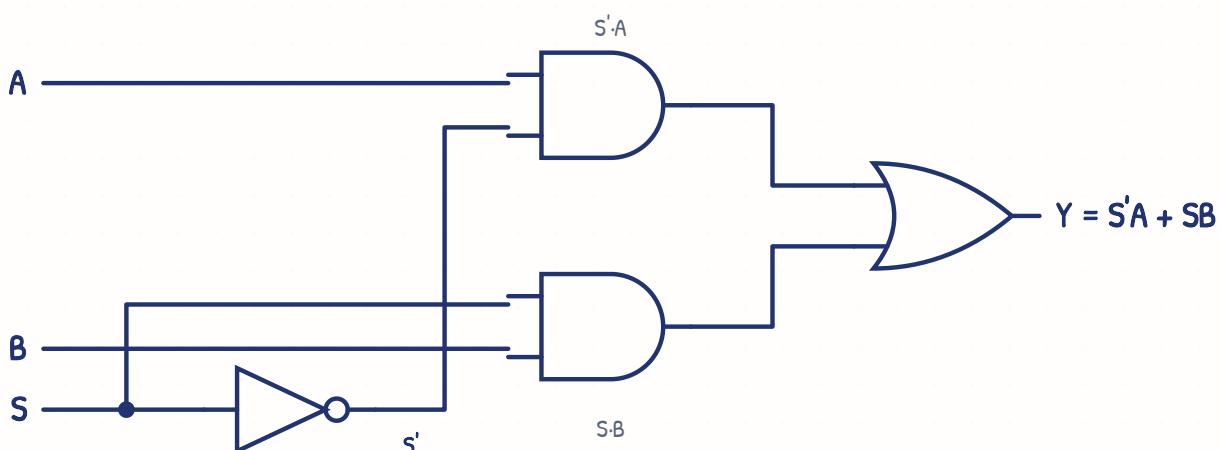
Hinglish: MUX ek railway switch jaisa hai — kai patriyaan aati hain par lever (select) decide karta hai ki kaunsi train aage jayegi. n select lines se 2^n inputs choose ho sakte hain: 1 select $\rightarrow$ 2 inputs, 2 select $\rightarrow$ 4, 3 select $\rightarrow$ 8. Bade MUX chhote MUX jod ke bante hain (tree). Aur ek trick: n -variable ka koi bhi function ek $2^n:1$ MUX se ban sakta hai — variables ko select pe lagao aur har input pe 0, 1, C ya C' daal do.

$$2:1 \rightarrow Y = S' \cdot I_0 + S \cdot I_1$$

$$4:1 \rightarrow Y = S_1' S_0' \cdot I_0 + S_1' S_0 \cdot I_1 + S_1 S_0' \cdot I_2 + S_1 S_0 \cdot I_3$$

n selects $\rightarrow 2^n$ inputs. $2^n:1$ from $2:1$ MUXes needs $2^n - 1$ of them ($4:1 \rightarrow 3$, $8:1 \rightarrow 7$).

Example: $F(A,B,C) = \sum m(1,3,5,6)$, $S_1 = A$, $S_0 = B \rightarrow$ for $AB = 00, 01, 10$ $F = C$; for $AB = 11$ $F = C' \rightarrow I_0 = C$, $I_1 = C$, $I_2 = C$, $I_3 = C'$



2:1 MUX from gates

How to draw: NOT on S, two ANDs (A with S' , B with S), one OR.

Trick: MUX = many-to-one (Many $\rightarrow$ 1). DEMUX = one-to-many.

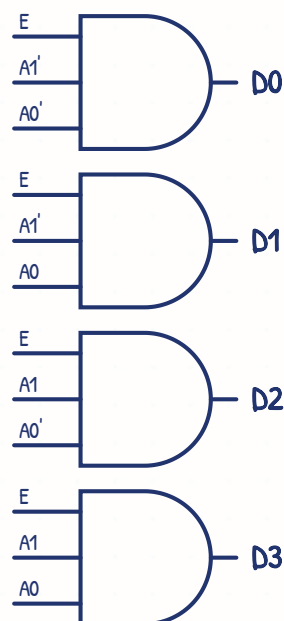
Yaad rakho: 8:1 MUX needs 3 select lines. Implementing a function = put variables on selects.

3.3 Demultiplexer and n-to-2ⁿ decoder

A **demultiplexer (DEMUX)** sends one data input to one of 2ⁿ outputs chosen by n select lines. A **decoder** converts an n-bit binary input into 2ⁿ outputs, exactly one of which is active (one-hot).

- 1:2 DEMUX: $Y_0 = S'D$, $Y_1 = SD$.
- Decoder output D_k = minterm k ; built from 2ⁿ AND gates (+ n inverters).
- A decoder = a DEMUX with its data input tied to 1 (enable).
- Any function = OR of the decoder outputs for its minterms (full adder: $\text{Sum} = D_1 + D_2 + D_4 + D_7$, $\text{Carry} = D_3 + D_5 + D_6 + D_7$).

	MUX	DEMUX	Decoder
In	2 ⁿ data + n select	1 data + n select	n bits
Out	1	2 ⁿ	2 ⁿ (one-hot)



E	A1	A0	D3	D2	D1	D0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

Each output = one minterm (one-hot).
Inverters for A1', A0' are shared.

2-to-4 decoder with enable

How to draw: Four 3-input ANDs: E with each combination of A1/A1' and A0/A0'.

Yaad rakho: 3-to-8 decoder = 8 AND gates, each output one minterm.

3.4 Half adder

A **half adder** is a combinational circuit that adds two single bits and produces a Sum and a Carry; it has no carry input.

Sum = $A \oplus B$, Carry = $A \cdot B$ ($1 + 1 = 10 \rightarrow$ Sum 0, Carry 1). One XOR + one AND.

Yaad rakho: no carry-in $\rightarrow$ cannot be chained for multi-bit addition.

3.5 Full adder & ripple-carry adder

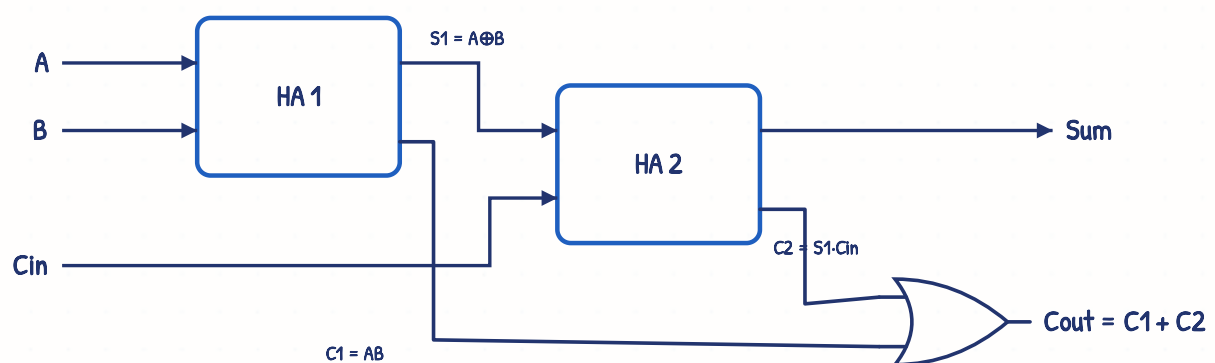
A **full adder** adds three bits — A, B and a carry-in C_{in} — and produces a Sum and a carry-out C_{out} . An **n-bit ripple-carry adder** chains n full adders, each carry-out feeding the next stage's carry-in.

Hinglish: Sum tab 1 hota hai jab teen mein se odd number of inputs 1 hon — isliye $Sum = A \oplus B \oplus C_{in}$. Carry tab 1 hota hai jab kam se kam do inputs 1 hon (majority). Full adder do half adders + ek OR se banta hai: pehla HA A aur B jodta hai, doosra HA us sum mein C_{in} jodta hai, aur dono carries ko OR kar dete hain (dono ek saath kabhi 1 nahi hote). Ripple adder slow hai kyunki har stage ko pichli stage ke carry ka wait karna padta hai.

Sum = $A \oplus B \oplus C_{in}$ $C_{out} = AB + C_{in}(A \oplus B) = AB + BC_{in} + AC_{in}$

Ripple delay grows **linearly**: ≈ 2 gate delays per bit (4-bit ≈ 8 , 64-bit ≈ 128).

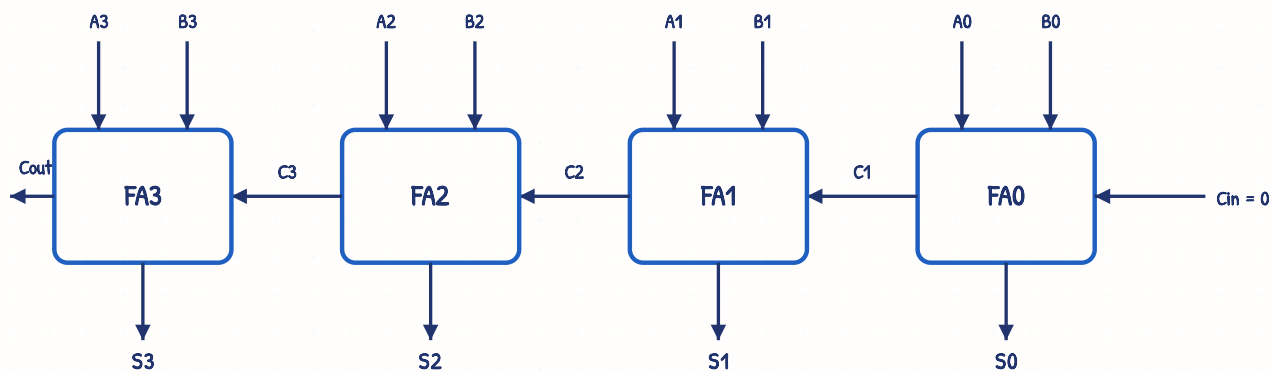
Example: $A = 1, B = 1, C_{in} = 1 \rightarrow Sum = 1 \oplus 1 \oplus 1 = 1, C_{out} = 1 \cdot 1 + \dots = 1 \rightarrow 1 + 1 + 1 = 11_2 = 3 \checkmark$



C1 and C2 are never 1 together (if $A = B = 1$ then $S1 = 0$), so an OR (or XOR) merges them.

Full adder from two half adders + OR

How to draw: Two HA boxes in series, OR their carries.



Carry ripples right $\rightarrow$ left; worst-case delay $\approx n \times t(\text{carry})$. 32-bit $\rightarrow$ 32 stages.

4-bit ripple-carry adder

How to draw: Four FA boxes right to left (FA0 on the right), carry arrows leftward, Cin = 0 at the right.

Trick: Sum = "odd parity", Cout = "majority vote".

Yaad rakho: FA = 2 HA + 1 OR. Ripple delay $\propto n$.

3.6 CPU preview – ALU and register file

The **ALU (Arithmetic Logic Unit)** is the combinational unit of the CPU that performs arithmetic (add, subtract) and logic (AND, OR, compare) operations on its operands. The **register file** is the small fast set of registers that supplies those operands.

- 1-bit ALU = AND + OR + full adder + MUX choosing the operation. Subtraction: invert B and set carry-in = 1.
- ALU control: 0000 AND, 0001 OR, 0010 add, 0110 sub, 0111 slt. Zero flag = 1 when the result is 0.
- Register file: 2 read ports + 1 write port (Unit 7).

Yaad rakho: add 0010, sub 0110.

3.E1 Carry-lookahead adder

A **carry-lookahead adder (CLA)** computes all carries directly from the inputs using generate and propagate signals, instead of waiting for the carry to ripple.

Generate $G = A \cdot B$ (makes a carry), Propagate $P = A \oplus B$ (passes a carry), $C(i+1) = G_i + P_i \cdot C_i$
 $C2 = G1 + P1G0 + P1P0C0$

Yaad rakho: CLA = faster, more area. Ripple = slower, less area.

Unit 4 · Number Systems & Two's Complement

4.1 Number systems – binary, decimal, hex

A **positional number system** with base (radix) r represents a value as a sum of digits each multiplied by a power of r according to its position. Binary uses base 2, decimal base 10, hexadecimal base 16.

Value = $\sum \text{digit} \times \text{base}^{\text{position}}$ (positions 0, 1, 2 from the right).

Binary $\rightarrow$ decimal: $101101 = 32 + 8 + 4 + 1 = 45$

Decimal $\rightarrow$ binary: divide by 2, write remainders, read **bottom-up**: $42 \rightarrow 101010$

Binary $\leftrightarrow$ hex: groups of 4 bits: $1011\ 0101 = B5 = 11 \times 16 + 5 = 181$ (A = 10 ... F = 15)

Yaad rakho: 1 hex digit = 4 bits. Powers of 2: 1 2 4 8 16 32 64 128 256 512 1024.

4.2 Signed representations

A **signed number representation** encodes both positive and negative integers in binary. The three schemes are sign-magnitude, one's complement and two's complement.

Scheme	Negative of x	Range (n bits)	Zeros
Sign-magnitude	flip the MSB	$\pm(2^{n-1} - 1)$	two
One's complement	invert all bits	$\pm(2^{n-1} - 1)$	two
Two's complement	invert + 1	-2^{n-1} to $2^{n-1} - 1$	one

Yaad rakho: computers use two's complement – one zero, and the same adder works for + and –.

4.3 Two's complement – negation & range

In **two's complement**, the most significant bit has weight -2^{n-1} and all other bits have their normal positive weights; the negative of a number is obtained by inverting all its bits and adding 1.

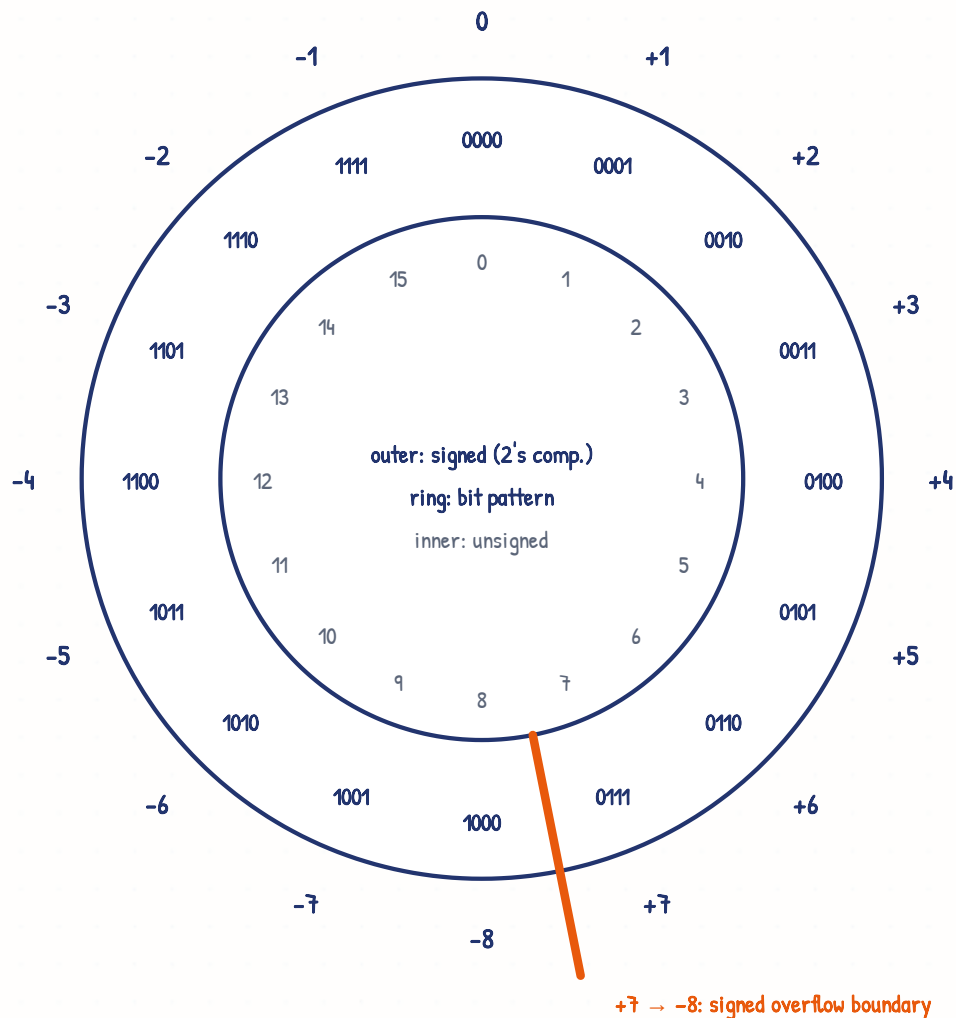
Hinglish: Kisi number ka negative chahiye toh do step: saare bits ulte karo ($0 \leftrightarrow 1$), phir 1 jod do. Jaise $+18 = 00010010 \rightarrow$ ulta $11101101 \rightarrow +1 = 11101110 = -18$. Check karne ka aasaan tareeka: MSB ka weight negative hota hai (8-bit mein -128), baaki sab positive. $11101110 = -128 + 64 + 32 + 8 + 4 + 2 = -18$. Range yaad rakho: negative side ek zyada hoti hai – 4-bit mein -8 se $+7$, 8-bit mein -128 se $+127$.

$-x = x' + 1$ · Range = -2^{n-1} to $2^{n-1} - 1$ · MSB weight = -2^{n-1}

4-bit $-8 \dots 7$ · 8-bit $-128 \dots 127$ · 16-bit $-32768 \dots 32767$

Example: $-45 \rightarrow 45 = 00101101 \rightarrow$ invert $11010010 \rightarrow +1 \rightarrow 11010011$. $0xB7 = 10110111 = -128 + 55 = -73$ (unsigned 183).

1111 → 0000 (-1 → 0): unsigned carry-out



4-bit number wheel (unsigned, pattern, signed)

How to draw: Circle with 16 positions; 0000 at the top, increase clockwise; label signed values outside and mark the +7/-8 boundary.

Trick: shortcut – from the right, copy bits up to the first 1, then flip the rest.

Yaad rakho: 1111...1 = -1; 1000...0 = most negative.

4.4 Sign extension

Sign extension is increasing the number of bits of a two's-complement number while keeping its value, by copying the sign bit (MSB) into all the new higher bits.

+5 = 0101 → 0000 0101 · -5 = 1011 → **1111 1011** (zero-padding would wrongly give +11)

MIPS: 16-bit immediate 0xFFFFB → 0xFFFFFFF B = -5.

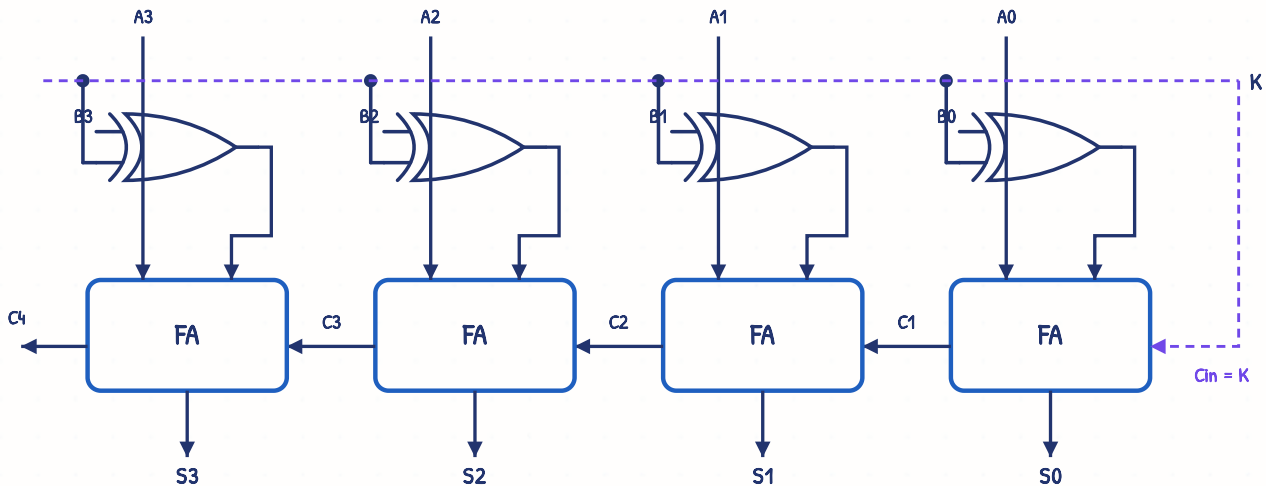
Yaad rakho: signed → copy MSB; unsigned (lbu, andi, ori) → fill with 0.

4.5 Binary addition & subtraction

Binary subtraction in two's complement is done by adding the two's complement of the subtrahend: $A - B = A + B' + 1$. An **adder-subtractor** uses XOR gates on B and a mode bit M to do both with one adder.

$A - B = A + B' + 1$. Adder-subtractor: $B_i \oplus M$, $C_{in} = M$ ($M = 0$ add, $M = 1$ subtract).

Example: $5 - 3 = 0101 + 1100 + 1 = 0101 + 1101 = 10010 \rightarrow$ drop carry $\rightarrow 0010 = 2$



$K = 0$: B passes, $C_{in} = 0 \rightarrow A + B$. $K = 1$: B inverted, $C_{in} = 1 \rightarrow A + B' + 1 = A - B$.

4-bit adder-subtractor (Lab 5)

How to draw: Four FA boxes; an XOR in front of each B input with the shared control K; K also feeds C_{in} of FA0.

Yaad rakho: subtract = invert B + carry-in 1.

4.6 Overflow detection

Overflow occurs when the true result of an arithmetic operation is outside the range that can be represented with the available bits. Signed overflow is flagged by V (OF), unsigned overflow by the carry flag C (CF).

Hinglish: Signed overflow sirf tab hota hai jab do same sign wale numbers jodne par answer ka sign ulta aa jaye — do positive jode aur answer negative aaya, ya do negative jode aur positive aaya. Alag sign wale numbers jodne par kabhi overflow nahi hota. Hardware mein check: MSB column mein aane wala carry (C_3) aur jaane wala carry (C_4) alag hon toh overflow. Unsigned ke liye sirf C_4 dekho.

$$V = C_{in(MSB)} \oplus C_{out(MSB)} = C_3 \oplus C_4 \text{ (4-bit)} \cdot CF = C_4$$

$$0111 + 0001 = 1000 \rightarrow 7 + 1 = -8 \rightarrow V = 1 (C_3 = 1, C_4 = 0)$$

$$1111 + 0001 = 0000 \rightarrow CF = 1, V = 0 \text{ (signed } -1 + 1 = 0 \text{ is fine)}$$

Yaad rakho: same signs in, different sign out $\rightarrow$ overflow. Opposite signs never overflow.

Unit 5 · Latches, Flip-Flops & Timing

5.1 Combinational vs sequential circuits

A **combinational circuit's** output depends only on its present inputs. A **sequential circuit's** output depends on present inputs AND past history (stored state), which requires memory elements built with feedback.

Combinational	Sequential
no memory	has memory (state)
no feedback	feedback loop
adder, MUX, decoder	latch, flip-flop, register, counter, FSM

Two inverters in a loop = **bistable** (two stable states) = simplest 1-bit memory.

Yaad rakho: sequential = combinational + memory (feedback).

5.2 SR latch & the forbidden state

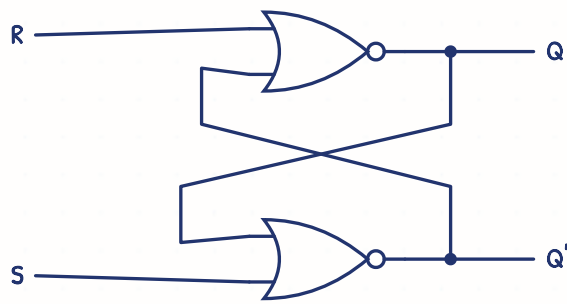
An **SR latch** is a basic memory element made of two cross-coupled NOR (or NAND) gates with Set and Reset inputs; Set makes $Q = 1$, Reset makes $Q = 0$, and with neither asserted it holds its previous value.

Hinglish: Socho ek bulb jiske do button hain — S dabao toh ON ($Q = 1$), R dabao toh OFF ($Q = 0$), kuch mat dabao toh jaisa tha waisa rahega (memory). Dono ek saath dabana galat hai: NOR latch mein dono outputs 0 ho jaate hain (Q aur Q' barabar — rule tootna hai), aur dono ek saath chhodo toh race hoti hai, result unpredictable. NAND latch active-LOW hota hai — wahan 0 dabane ka matlab hai, aur 00 forbidden hai.

NOR: $S R = 00$ hold · 10 set · 01 reset · **11 forbidden** ($Q = Q' = 0$)

NAND (active-low): 11 hold · **00 forbidden** ($Q = Q' = 1$)

Characteristic equation: $Q^+ = S + R'Q$, with $SR = 0$



S	R	Q ⁺	action
0	0	Q	hold
1	0	1	set
0	1	0	reset
1	1	-	forbidden (Q=Q'=0)

NOR latch: inputs are active-HIGH. Each output feeds the other gate.

NOR SR latch + characteristic table

How to draw: Two NOR gates stacked; draw the output of each crossing to the second input of the other; R on the gate that makes Q.

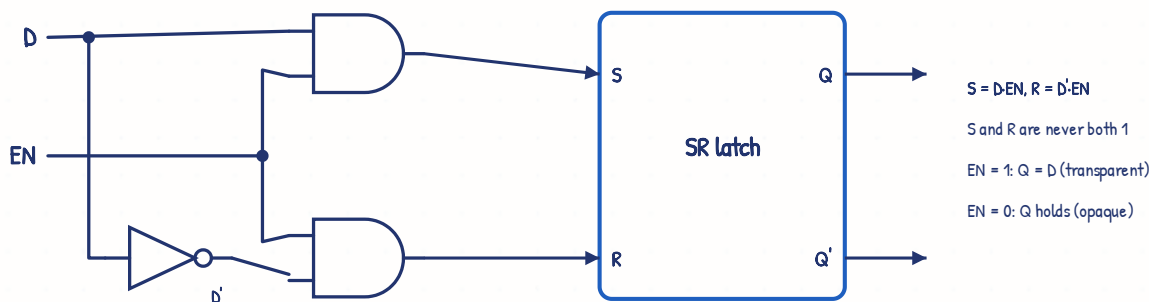
Yaad rakho: NOR latch: 11 forbidden. NAND latch: 00 forbidden.

5.3 D latch (level-triggered)

A **D latch** is a gated latch with one data input D and an enable EN: while EN = 1 it is transparent (Q follows D); while EN = 0 it holds the last value. It is level-triggered.

- Built as $S = D \cdot EN$, $R = D' \cdot EN \rightarrow$ S and R can never both be 1, so no forbidden state.
- Glitches on D pass to Q while EN = 1 $\rightarrow$ unsafe for pipelines.

$$Q^+ = EN \cdot D + EN' \cdot Q$$



D latch from an SR latch

How to draw: Inverter on D, two AND gates with EN, feeding S and R of an SR latch box.

Yaad rakho: latch = level (window open), flip-flop = edge (camera click).

5.4 D flip-flop (edge-triggered)

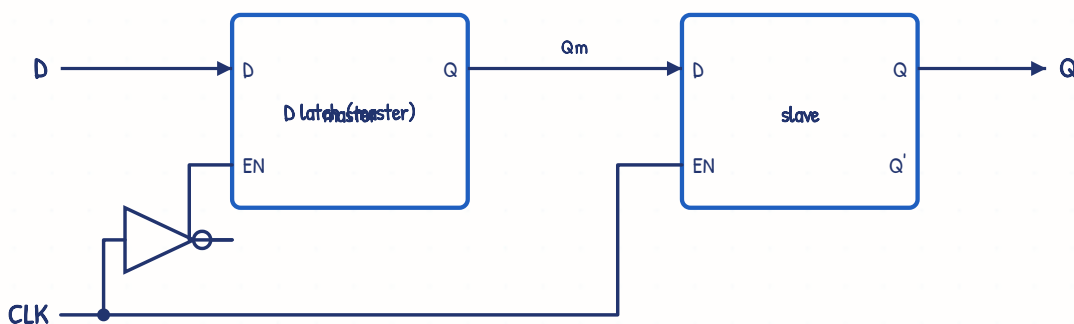
A **D flip-flop** is an edge-triggered memory element that copies its D input to Q only at the active clock edge (usually rising) and holds Q constant at all other times.

Hinglish: Latch khidki jaisa hai — jab tak khuli hai ($EN = 1$) bahar ka sab andar dikhta hai. Flip-flop camera jaisa hai — sirf click (clock edge) ke instant ki photo leta hai, beech mein kuch bhi ho, Q nahi badalta. Andar se flip-flop do latches ka bana hota hai (master-slave): ek ki khidki khuli toh doosre ki band, isliye data sirf edge pe hi aage jaata hai. Waveform mein har rising edge pe line kheencho, wahan D ki value padho, aur agle edge tak wahi Q rakho.

At the rising edge: $Q^+ = D$. Verilog: always @(posedge clk) q <= d;

Master-slave: two D latches on opposite clock phases. Master open when $CLK = 0 \rightarrow Q$ changes at the RISING edge (class slide version: master open at $CLK = 1 \rightarrow$ falling edge).

Clock frequency $f = 1/T$; ring oscillator (odd number N of inverters) $T = 2 \cdot N \cdot t_{pd}$.



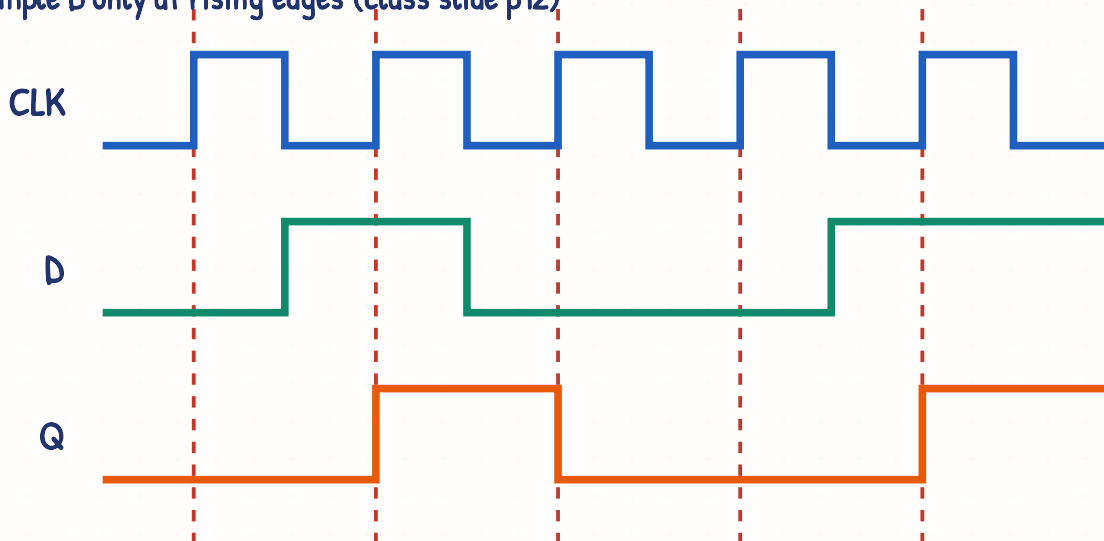
$CLK = 0$: master transparent (Q_m follows D), slave holds. $CLK 0 \rightarrow 1$: master closes, slave opens $\rightarrow Q$ takes D at the RISING edge.

Class slide variant: master enabled on $CLK = 1 \rightarrow$ the output changes at the FALLING edge (negative-edge FF).

Master-slave D flip-flop

How to draw: Two D-latch boxes in series; CLK to one enable directly and to the other through an inverter.

Trace Q: sample D only at rising edges (class slide p12)



D flip-flop waveform trace (class slide)

How to draw: Draw dashed lines at every rising edge; read D there and hold that value until the next edge.

Yaad rakho: FF = 2 latches, $\approx 2 \times$ gates, edge-triggered, used for every register and pipeline stage.

5.5 Setup time, hold time, metastability

Setup time is the minimum time the D input must be stable BEFORE the clock edge; **hold time** is the minimum time it must stay stable AFTER the edge; **clock-to-Q delay (t_{pcq})** is the time from the edge until Q is valid. **Metastability** is the state in which a flip-flop's output stays between 0 and 1 for an unpredictable time because setup or hold was violated.

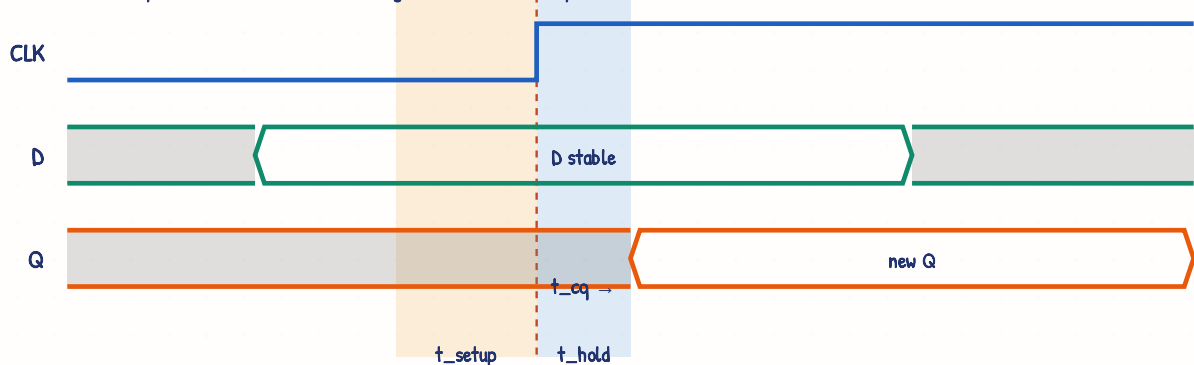
Hinglish: Photo khinchte waqt object ko click se thoda pehle (setup) aur thoda baad (hold) tak hilna nahi chahiye, warna photo blur. Waise hi D ko edge ke aas-paas stable rehna chahiye. Clock kitna fast ho sakta hai: data ko pehle flip-flop se nikalna hai (t_{pcq}), logic se guzarna hai (t_{pd}), aur agle edge se setup time pehle pahunchna hai — teeno ka sum hi minimum clock period hai. Hold violation clock slow karke theek nahi hota, short path mein delay daalna padta hai. Metastability ko poori tarah hata nahi sakte, sirf 2-3 flip-flop synchronizer se bahut rare kar sakte hain.

$$T_c \geq t_{pcq} + t_{pd} + t_{setup} \rightarrow f_{max} = 1 / T_c$$

Hold: $t_{ccq} + t_{cd} \geq t_{hold}$ (independent of the clock)

Example: 0.28 ns path + 0.05 ns setup $\rightarrow T = 0.33$ ns $\rightarrow f = 1/0.33$ ns $\approx$ **3.03 GHz** ($1/\text{ns} = 1$ GHz)

D must be stable from t_{setup} before to t_{hold} after the edge; Q valid $t_{clk-to-q}$ later



Setup, hold and clock-to-Q on a timing diagram

How to draw: One rising edge; shade the window before (setup) and after (hold) where D must be flat; Q changes t_{cq} after the edge.

Yaad rakho: setup = before, hold = after. Slow clock fixes setup, NOT hold. Metastability $\rightarrow$ synchronizer.

5.E1 JK and T flip-flops

A **JK flip-flop** is like SR but $J = K = 1$ toggles the output. A **T (toggle) flip-flop** toggles when $T = 1$ and holds when $T = 0$.

JK: 00 hold, 01 reset, 10 set, 11 toggle $\rightarrow Q^+ = JQ' + K'Q$

T: $Q^+ = T \oplus Q$. T from D: $D = T \oplus Q$. T from JK: $J = K = T$.

Excitation: $D = Q^+$, $T = Q \oplus Q^+$ (1 when the bit must change).

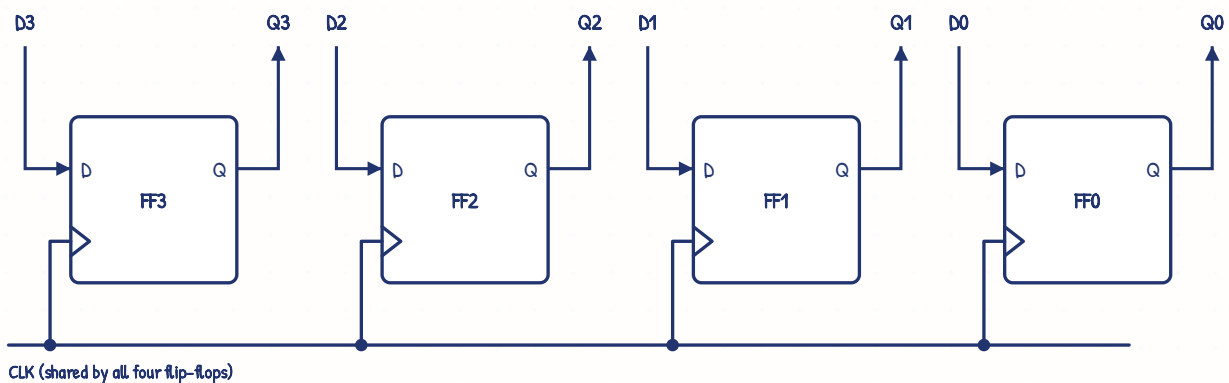
Yaad rakho: SR $Q^+ = S + R'Q$. D $Q^+ = D$. JK $Q^+ = JQ' + K'Q$. T $Q^+ = T \oplus Q$.

Unit 6 · Registers, Counters & Shift Registers

6.1 From flip-flops to registers

A **register** is a group of n flip-flops sharing a common clock that stores an n -bit word, all bits being loaded at the same clock edge.

- n -bit register = n D flip-flops + one common clock.
- PC, IR, general registers, pipeline registers — all registers.



A register = n D flip-flops sharing one clock; all n bits are captured at the same rising edge.

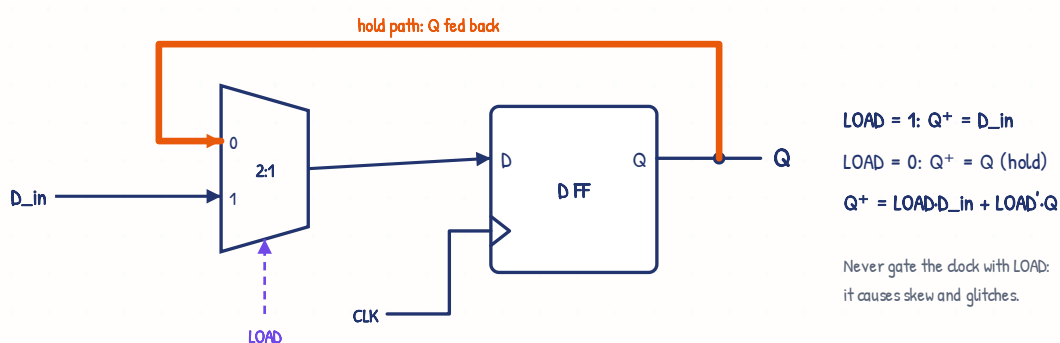
4-bit register (4 D flip-flops, one clock)

How to draw: Four D-FF boxes in a row, D inputs above/left, Q outputs right, one horizontal clock line underneath tapping each clock pin.

6.2 Load enable & reset

Load enable lets a register either load new data or keep its value at a clock edge, using a 2:1 MUX in front of each flip-flop. **Reset** forces the register to a known value (0): **synchronous reset** acts at the clock edge, **asynchronous reset** acts immediately.

$Q^+ = \text{LOAD} \cdot D + \text{LOAD}' \cdot Q$ (LOAD = 0 feeds Q back $\rightarrow$ hold). Priority in labs: **reset** > **load** > **hold**.



Load enable: 2:1 MUX in front of each D

How to draw: Trapezoid MUX feeding a D FF; input 1 = new data, input 0 = wire looped back from Q; LOAD on the MUX select.

Yaad rakho: never gate the clock with LOAD (glitches, skew). Use the MUX.

6.3 Ripple (asynchronous) vs synchronous counters

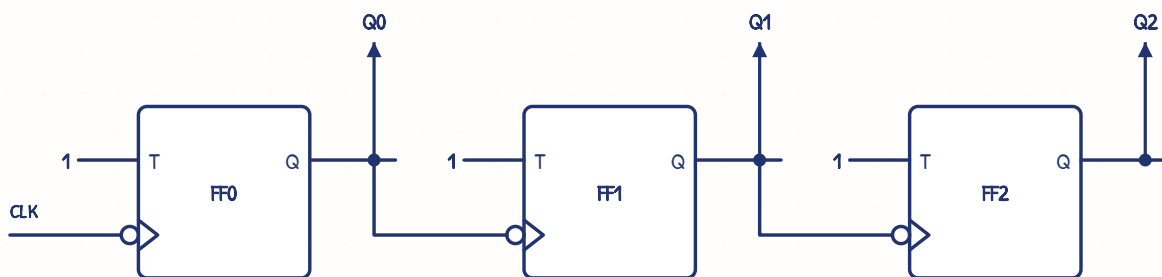
A **counter** is a sequential circuit that goes through a fixed sequence of states (usually binary counting) on successive clock pulses. In a **ripple counter** only the first flip-flop gets the clock and each next one is clocked by the previous output; in a **synchronous counter** all flip-flops share the same clock.

Hinglish: Binary counting dekho: koi bhi bit tab badalti hai jab uske neeche ke saare bits 1 hon (0111 → 1000). Ripple counter mein domino ki tarah ek ke baad ek bit girti hai, isliye time lagta hai ($n \times t_{pd}$) aur beech mein galat values dikhti hain: 7 → 6 → 4 → 0 → 8. Synchronous counter mein sab bits ek saath, ek hi clock pe badalti hain — fast aur glitch-free, bas thode AND gates extra lagte hain.

Ripple delay = $n \times t_{pd}$, $f_{max} = 1 / (n \times t_{pd})$

Bit k frequency = $f / 2^{(k+1)}$ ($Q_0 = f/2$, $Q_1 = f/4$)

Example: 4-bit, $t_{pd} = 10 \text{ ns} \rightarrow 40 \text{ ns} \rightarrow f = 1000/40 \text{ MHz} = 25 \text{ MHz}$



Only FF0 sees the clock. Each next FF is clocked by the previous Q (bubble = falling-edge trigger), so FF1 toggles when Q0 falls 1 → 0. Bits change one after another: settle time = $n \times t_{pd}$.

3-bit ripple counter (T flip-flops, T = 1)

How to draw: Three T-FF boxes, T tied to 1; clock only into FF0; draw each Q both upward (output) and into the next FF's clock with a bubble.

	Ripple	Synchronous
Clock	FF0 only	shared
Speed	slow ($n \cdot t_{pd}$)	fast
Glitches	yes	no
Hardware	less	more (AND gates)

Yaad rakho: ripple = dominoes, synchronous = soldiers marching together.

6.4 4-bit synchronous up-counter design

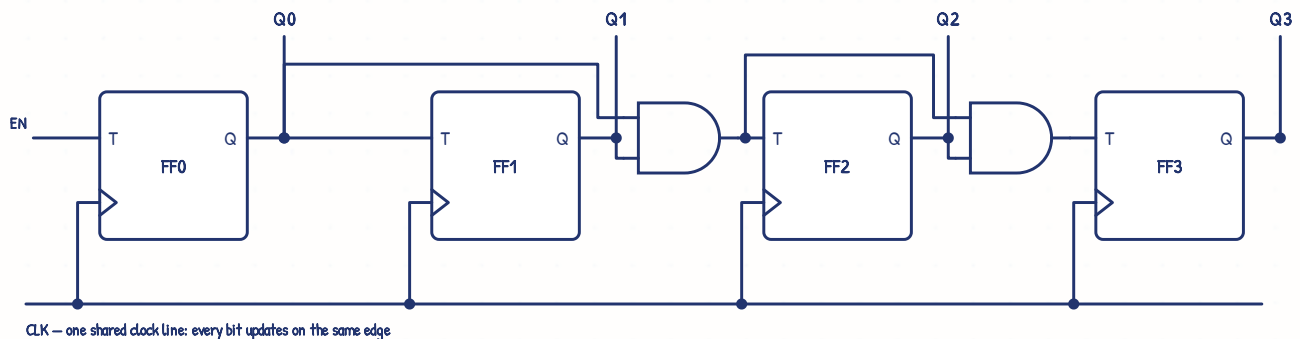
A **synchronous up-counter** is designed by writing its state table, finding each flip-flop input from the excitation table, and minimising with K-maps.

T flip-flops: $T_0 = 1, T_1 = Q_0, T_2 = Q_0 \cdot Q_1, T_3 = Q_0 \cdot Q_1 \cdot Q_2$

D flip-flops: $D_0 = Q_0', D_1 = Q_1 \oplus Q_0, D_2 = Q_2 \oplus Q_1 Q_0$

Down counter: $T_1 = Q_0', T_2 = Q_0' Q_1'$

Flip-flops for mod-N = $\lceil \log_2 N \rceil$ = smallest k with $2^k \geq N$ (mod-12 $\rightarrow$ 4, mod-1000 $\rightarrow$ 10)



$T_0 = EN (1), T_1 = Q_0, T_2 = Q_0 \cdot Q_1, T_3 = Q_0 \cdot Q_1 \cdot Q_2$ — a bit toggles when all lower bits are 1.

4-bit synchronous up-counter (T flip-flops + AND chain)

How to draw: Four T FFs in a row on one clock line; $T_0 = EN$; T_1 from Q_0 ; an AND gate (Q_0, Q_1) feeds T_2 ; a second AND (previous AND, Q_2) feeds T_3 .

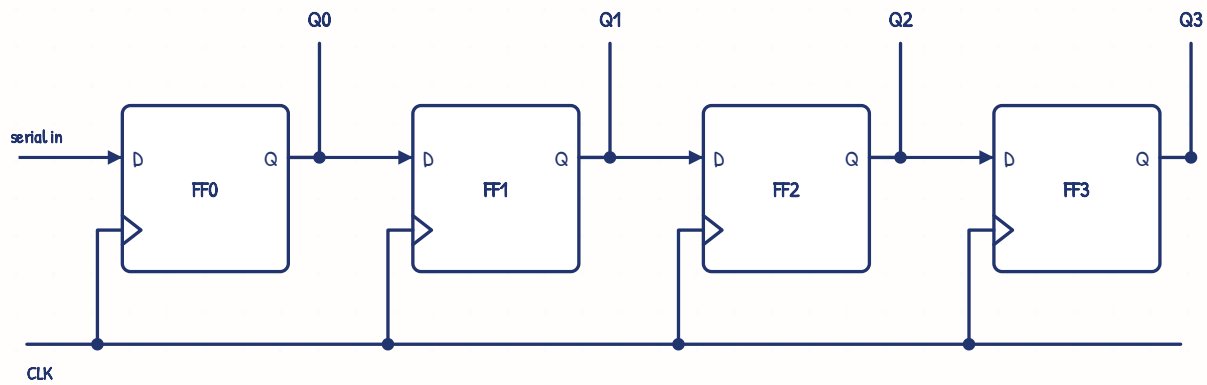
Trick: "a bit flips when everything below it is 1".

Yaad rakho: T-chain = AND of all lower Q 's.

6.5 Shift registers: SISO, SIPO, PISO, PIPO

A **shift register** is a chain of flip-flops on a common clock in which each flip-flop's output feeds the next one's input, so the stored bits move one position per clock pulse.

Type	In $\rightarrow$ Out	Use
SISO	serial $\rightarrow$ serial	delay line
SIPO	serial $\rightarrow$ parallel	UART receiver
PISO	parallel $\rightarrow$ serial	UART transmitter (needs load MUX)
PIPO	parallel $\rightarrow$ parallel	buffer / register



SIPO: same chain, every Q tapped. After 4 clocks the whole word is on Q0-Q3 at once (UART receiver).

SIPO (4-bit)

How to draw: Same as SISO, then draw a vertical tap from every Q up to labelled outputs Q0-Q3.

Yaad rakho: receive = SIPO, transmit = PISO.

6.E1 Ring, Johnson, mod-N, LFSR

A **ring counter** is a shift register whose last output feeds the first input (one circulating 1). A **Johnson counter** feeds back the complement of the last output. An **LFSR** feeds back the XOR of selected bits to generate pseudo-random sequences.

Ring: **n states** · Johnson: **2n states** (unused $2^n - 2n$) · LFSR (maximal): **$2^n - 1$ states** · binary: 2^n

Yaad rakho: 4-bit ring 4 states, Johnson 8, binary 16, LFSR 15.

Unit 7 · Register Files & Finite State Machines

7.1 From registers to register files

A **register file** is an array of registers inside the CPU with address-based access, allowing several registers to be read and one to be written in the same clock cycle.

- Need: add R1, R2, R3 reads 2 and writes 1 register in one cycle.
- MIPS/RISC-V: 32 registers × 32 bits, 2 read ports + 1 write port.

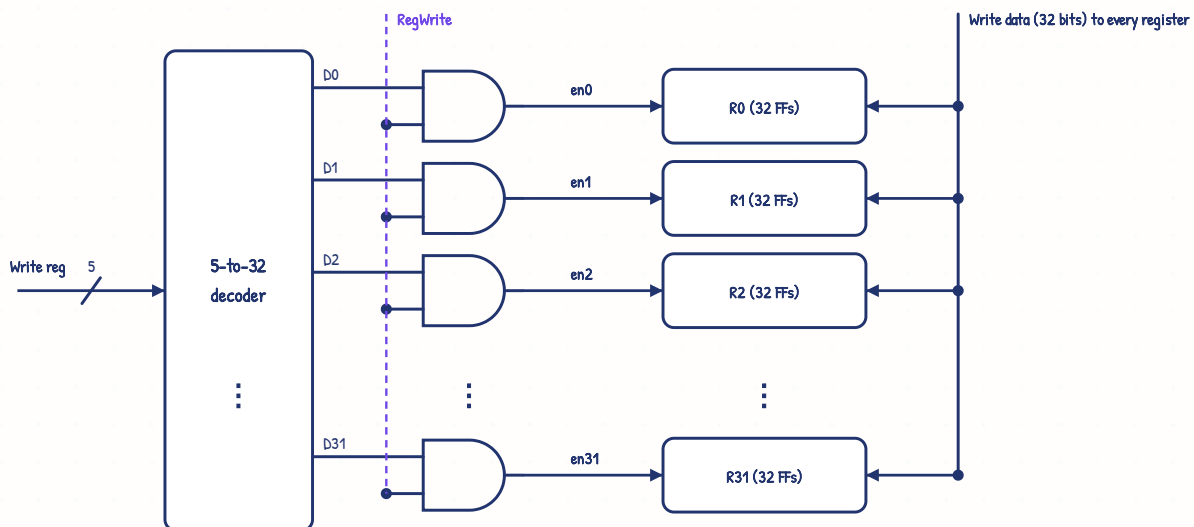
Address bits = $\log_2(\text{number of registers})$ (32 → 5) · flip-flops = registers × width (16 × 8 = 128)

7.2 Register file architecture (1 write, 2 read)

The **write port** of a register file uses a decoder plus a write-enable signal so that exactly one register is loaded at the clock edge; each **read port** is a multiplexer that selects one register's contents using the read address.

Hinglish: Likhne ke liye: 5-bit address decoder mein jaata hai, decoder 32 mein se sirf ek line ON karta hai (kaunsa register), aur RegWrite batata hai likhna hai ya nahi — dono ka AND us register ka load enable hai. Padhne ke liye: har register ka output do bade 32:1 MUX mein jaata hai, address MUX ka select hai. Read ke liye clock ki zarurat nahi (combinational, turant), write sirf clock edge pe (synchronous) — isse ek hi cycle mein purani value padh ke nayi likh sakte hain.

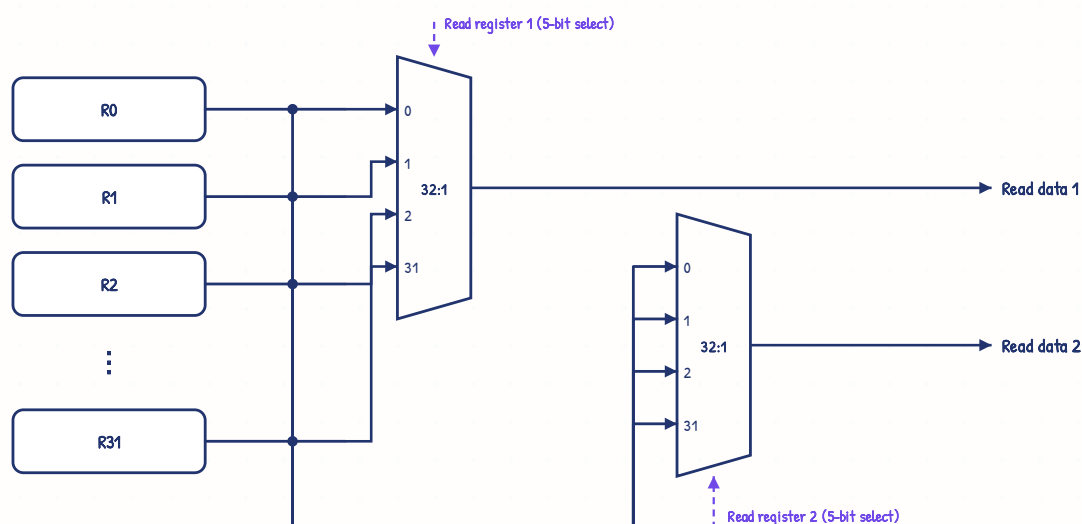
- Write: decoder picks WHERE, RegWrite picks WHETHER.
- Read: two 32:1 MUXes, combinational (no clock); write: synchronous.
- Register 0 (\$zero / x0) always reads 0, writes ignored.



The decoder picks WHERE (one-hot), RegWrite picks WHETHER. Only the enabled register captures the data at the edge.

Write port: 5-to-32 decoder + RegWrite → register enables

How to draw: Tall decoder box; each output into an AND gate with RegWrite; AND output = Load enable of R0..R31; one Write data bus to all registers.



Every register output fans out to BOTH 32:1 MUXes (wide 32-bit buses, drawn as single lines). No clock: data appears after gate delay.

Read ports: two 32:1 MUXes

How to draw: Stack of register boxes on the left; every output goes to both MUXes; Read reg 1 / 2 are the MUX selects; outputs Read data 1 / 2.

Yaad rakho: read = MUX = combinational; write = decoder + WE = clocked.

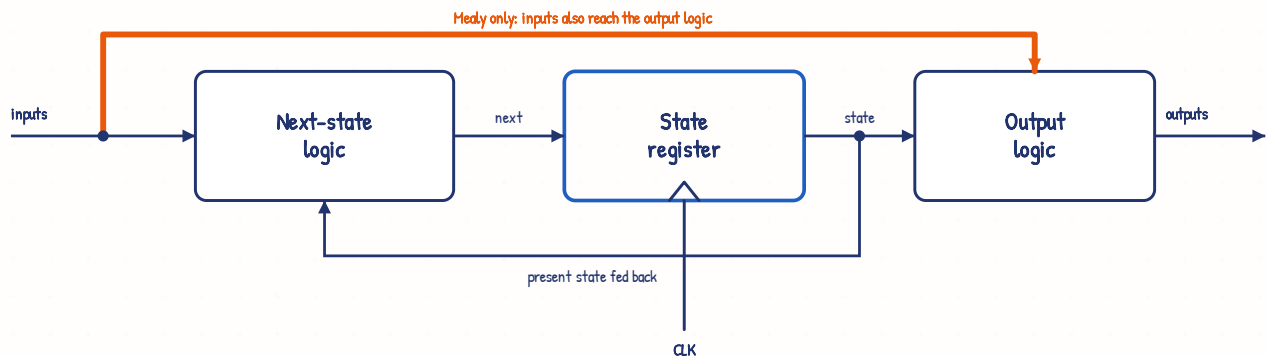
7.3 Finite state machines (FSM)

A **finite state machine** is a sequential circuit with a finite number of states that moves from one state to another according to its inputs and produces outputs; formally $(S, I, O, \delta, \lambda)$ where δ is the next-state (transition) function and λ the output function.

- Hardware: next-state logic → state register (flip-flops) → output logic, with feedback.

- Examples: traffic light, vending machine, the CPU control unit.
- Design steps: spec → state diagram → state table → encode → equations → circuit.

Flip-flops: binary encoding $\lceil \log_2 N \rceil$, one-hot N



Moore: output = $\lambda(\text{state})$. Mealy: output = $\lambda(\text{state}, \text{input})$. Next state = $\delta(\text{state}, \text{input})$ for both.

FSM anatomy (Moore and Mealy)

How to draw: Three boxes in a row (next-state logic, state register with clock, output logic), feedback arrow from the register back to next-state logic; for Mealy add an arrow from the inputs to the output logic.

Yaad rakho: FSM = states + inputs + transitions + outputs; δ = next state, λ = output.

7.4 Moore vs Mealy

In a **Moore machine** the outputs depend only on the present state; in a **Mealy machine** the outputs depend on the present state and the present inputs.

Moore	Mealy
output = $f(\text{state})$	output = $f(\text{state}, \text{input})$
written inside the circle	written on the arrow (in/out)
output one cycle later	same-cycle output
glitch-free	can glitch
more states	fewer states

Trick: Moore = output in the circle (O); Mealy = output on the arrow.

7.5 Sequence detector 1011

A **sequence detector** is an FSM that outputs 1 when a specific bit pattern has appeared in its serial input. With **overlap**, the end of one match may start the next; without overlap, the machine restarts after a match.

Hinglish: Har state yaad rakhti hai ki ab tak pattern ka kitna hissa match hua: S_0 = kuch nahi, S_1 = "1", S_2 = "10", S_3 = "101". Galat bit aaye toh S_0 pe mat jao — dekho ki ab tak ke input ka aakhri hissa pattern ki

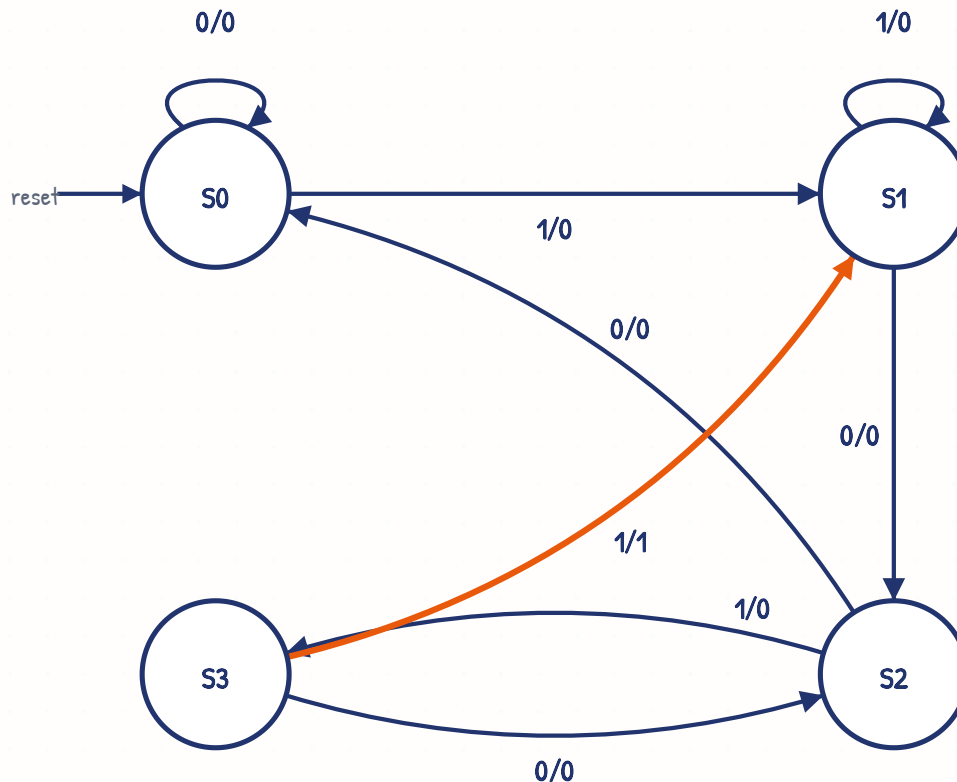
shuruaat se kitna milta hai. Jaise S3 ("101") pe 0 aaya → "1010" → aakhir mein "10" bacha → S2. Match hone pe overlap allowed ho toh aakhri "1" ko reuse karo (S1), nahi toh S0. Moore mein output ke liye ek extra state S4 lagti hai, isliye Moore = 5 states, Mealy = 4.

Mealy: S3 → 1/1 → S1 (overlap) or S0 (no overlap). 4 states → 2 FFs.

Moore: S4 (out 1); S4 → 1 → S1, → 0 → S2 (overlap) or S0 (no overlap). 5 states → 3 FFs (5 one-hot).

Stream 10110110: overlap → 2 matches; no overlap → 1 match.

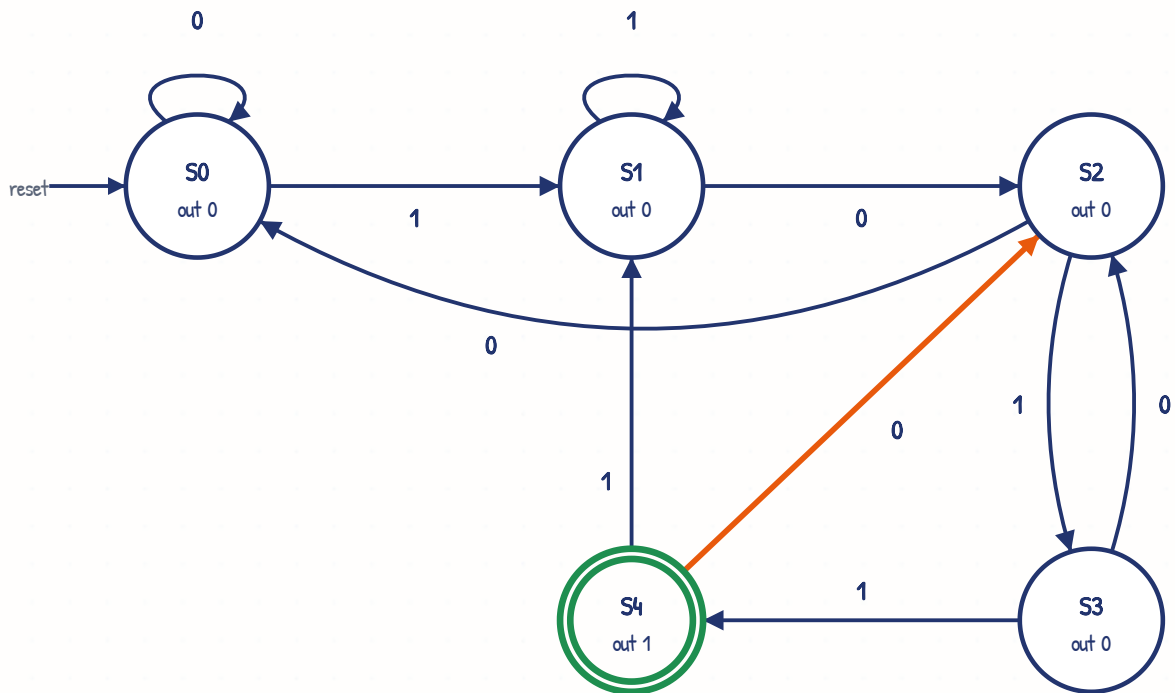
Mealy 1011, overlap: S3 → 1/1 → S1 (input/output on arrows)



Mealy 1011 detector, overlapping

How to draw: Four circles S0–S3 in a square; forward edges 1/0, 0/0, 1/0; fall-back edges; the amber S3 → S1 edge labelled 1/1.

Moore 1011, overlap: $S_4 \xrightarrow{0} S_2, \xrightarrow{1} S_1$ (output in the circle)



Moore 1011 detector, overlapping (5 states)

How to draw: Five circles; output written inside ($S_4 = 1$); S_4 on 1 $\rightarrow$ S_1 , on 0 $\rightarrow$ S_2 .

Yaad rakho: never jump to S_0 after a match when overlap is allowed.

Unit 8 · SRAM vs DRAM & the Memory Stack

8.1 Why we need bigger memory

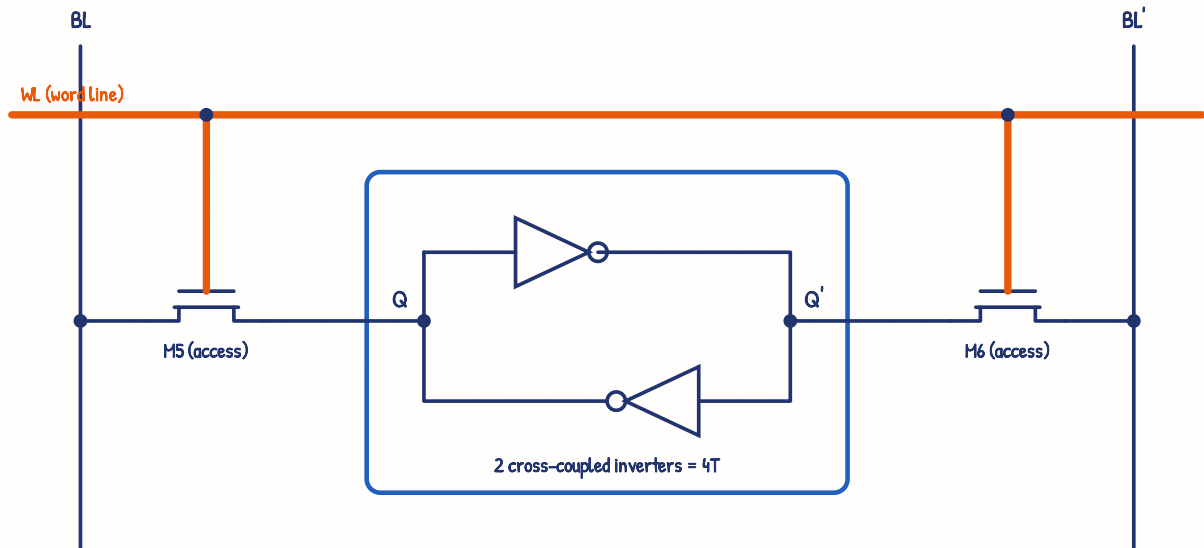
Register files are fast but tiny because each bit uses a flip-flop (20+ transistors); larger memories use denser cells — SRAM and DRAM — trading speed for capacity.

Yaad rakho: fast memory is small and expensive; big memory is slow and cheap.

8.2 SRAM — the 6T cell

SRAM (Static RAM) stores each bit in a bistable latch of two cross-coupled inverters (4 transistors) plus two access transistors — a 6-transistor cell — and keeps the data without refresh as long as power is on.

- Word line (WL) selects the cell; bit lines BL and BL' carry data and its complement.
- Read: precharge both bit lines, raise WL, sense amplifier detects which line drops — **non-destructive**.
- Write: drive bit lines, raise WL, drivers overpower the latch.
- Fast (few ns), low density, expensive → **caches, register files**.



4 transistors (2 inverters, each 1 PMOS + 1 NMOS) store the bit actively; WL = 1 connects Q to BL and Q' to BL'.

Total 6T per bit. Reads are non-destructive; the latch holds the value as long as power is on (static).

6T SRAM cell

How to draw: Two vertical bit lines (BL, BL'), a word line across the top; in the middle two inverters drawn back-to-back in a loop; an access transistor on each side between the loop node (Q / Q') and its bit line, gates tied to WL.

Yaad rakho: 6T = 4 (latch) + 2 (access). Static = no refresh.

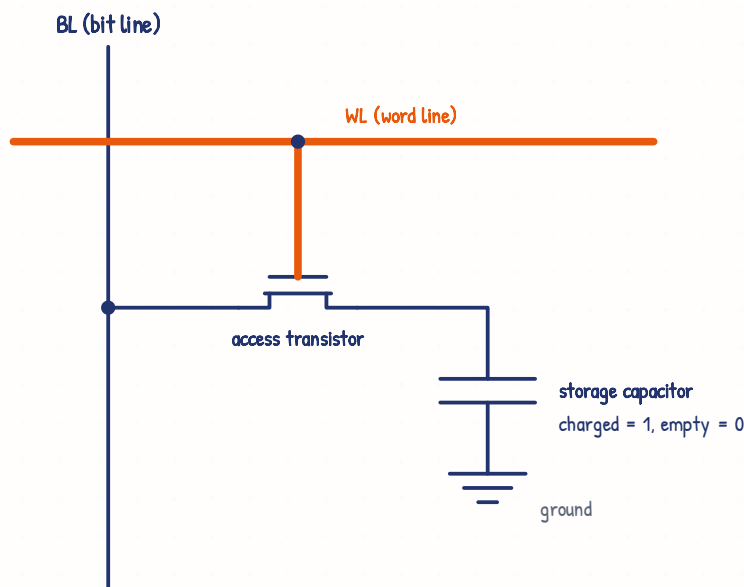
8.3 DRAM – 1T1C cell, refresh, destructive read

DRAM (Dynamic RAM) stores each bit as charge on a capacitor accessed through one transistor (1T1C); because the charge leaks, every row must be periodically refreshed, and a read drains the capacitor so the value must be written back.

Hinglish: DRAM mein bit ek chhote capacitor mein charge ki tarah rehta hai — charge hai toh 1, nahi toh 0. Capacitor dheere-dheere leak hota hai (lagbhag 64 ms mein data khatam), isliye controller har row ko baar-baar padh ke wapas likhta hai — isi ko refresh kehte hain, aur isi wajah se naam “dynamic”. Padhte waqt capacitor ka charge bit line mein bant jaata hai, toh value mit jaati hai (destructive read) — sense amplifier usse pehchaan ke turant wapas likhta hai. Sirf 1 transistor + 1 capacitor, isliye SRAM se ~6 guna dense aur sasta.

Refresh: every row about every **64 ms**. **Per-row interval = retention time ÷ rows**

Example: $64 \text{ ms} \div 8192 \text{ rows} = 0.0078125 \text{ ms} = 7.8 \mu\text{s}$



WL = 1 opens the transistor and connects the capacitor to the bit line (write: charge/drain it; read: share its charge).

1 transistor + 1 capacitor per bit → about 6× denser than SRAM, but the charge leaks (needs refresh).

1T1C DRAM cell

How to draw: Vertical bit line, horizontal word line; one transistor from the bit line to a capacitor whose other plate goes to ground; gate tied to the word line.

Yaad rakho: DRAM = 1T1C, refresh, destructive read, 50–100 ns, main memory.

8.4 SRAM vs DRAM

SRAM vs DRAM: SRAM stores a bit actively in a 6-transistor latch (fast, no refresh, low density), while DRAM stores it passively as charge on a 1T1C cell (slower, needs refresh, high density and cheap).

	SRAM	DRAM
Cell	6T	1T1C
Storage	active latch	passive charge
Density	low	high (~6×)
Speed	few ns	50–100 ns
Refresh	no	yes
Read	non-destructive	destructive
Cost/bit	high	low
Use	cache	main memory

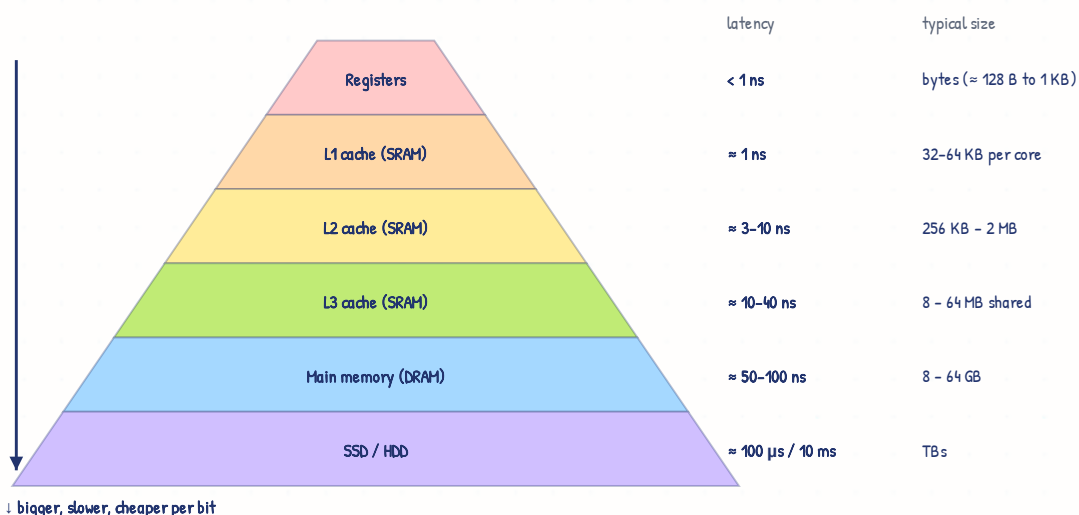
Yaad rakho: SRAM trades density for speed; DRAM trades speed for density.

8.5 Memory hierarchy & the memory wall

The **memory hierarchy** arranges storage in levels — registers, caches, main memory, disk — from small/fast/expensive to large/slow/cheap, exploiting **locality** so most accesses hit the fast levels. The **memory wall** is the growing gap between processor speed and memory (DRAM) latency.

- Order: registers < L1 < L2 < L3 < DRAM < SSD < HDD (fastest → slowest).
- **Temporal locality:** recently used data is used again soon. **Spatial locality:** nearby data is used soon.
- Memory wall: CPU speed grew ~50 %/yr, DRAM latency only ~7 %/yr.

AMAT = hit time + miss rate × miss penalty · Example $1 + 0.05 \times 80 = 5$ ns



Locality (temporal + spatial) lets the small fast levels serve most accesses — the system looks as fast as the top and as big as the bottom.

Memory hierarchy pyramid

How to draw: Triangle split into six bands (registers at the narrow top, disk at the wide base); write latency and size beside each band; arrow downward “bigger, slower, cheaper”.

Unit 9 · Von Neumann vs Harvard

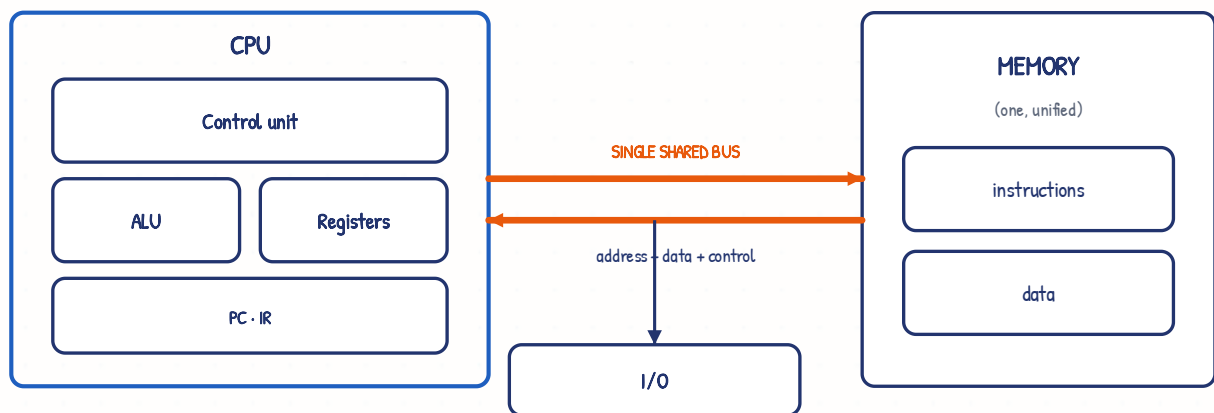
9.1 From gates to a complete machine

A complete computer consists of a **CPU** (compute), **memory** (store), **I/O** (communicate) and a **bus** (address, data and control lines) connecting them; **computer organisation** decides how they are connected.

9.2 Von Neumann architecture

The **Von Neumann (stored-program) architecture** keeps program instructions and data in the same memory and moves both over a single shared bus between memory and CPU. Described by John von Neumann in the "First Draft of a Report on the EDVAC" (1945).

- Four classic components: control unit, memory, ALU, I/O.
- **Stored-program concept**: programs are data – change the program by loading new numbers (enables compilers, loaders, JIT; also security risks).
- Won because of simplicity and generality.



Instructions and data take turns on the one bus – the defining feature and the built-in limit.

Von Neumann block diagram

How to draw: CPU box (control unit, ALU, registers) on the left, one memory box holding both instructions and data on the right, a single two-way bus between them; I/O on the bus.

9.3 The Von Neumann bottleneck

The **Von Neumann bottleneck** (term by John Backus, 1977) is the limit on performance caused by instructions and data sharing one bus, so an instruction fetch and a data access cannot happen at the same time.

Hinglish: Ek hi sadak (bus) pe instructions aur data dono chalte hain, toh ek waqt mein ek hi ja sakta hai — ya instruction lao ya data. CPU bahut tez hai par sadak utni tez nahi, isliye CPU wait karta rehta hai. Solution: cache, chaudi/tez bus, ya Harvard jaisa alag rasta.

Bandwidth = bits per cycle × clock frequency ÷ 8 (bytes/s)

Example: 1 GHz, 32-bit instr + 32-bit data → 64×10^9 bits/s ÷ 8 = **8 GB/s**

Backus (1977) called this traffic jam the “von Neumann bottleneck”: the CPU computes faster than one bus can feed it.

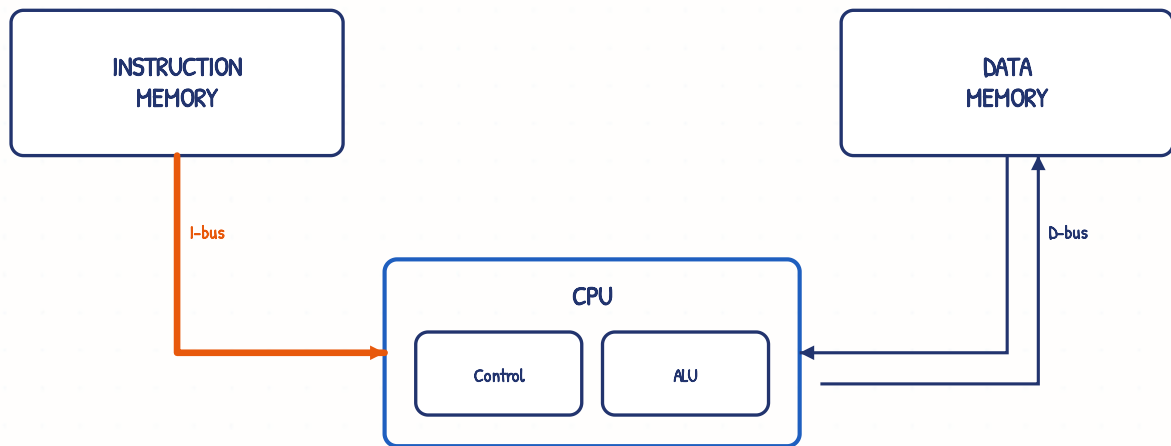
The single-bus bottleneck

How to draw: CPU and memory boxes joined by a bus drawn pinched in the middle; label “fetch OR data, never both”.

9.4 Harvard architecture

The **Harvard architecture** uses physically separate memories and buses for instructions and data, so an instruction fetch and a data access can happen in the same cycle. Named after the Harvard Mark I (1944).

	Von Neumann	Harvard
Memory / bus	one / one	two / two
Fetch + data	2 cycles	1 cycle
Throughput	bottlenecked	up to 2×
Cost	low	higher
Flexibility	high	lower
Use	general CPUs	DSPs, microcontrollers (ATmega328)



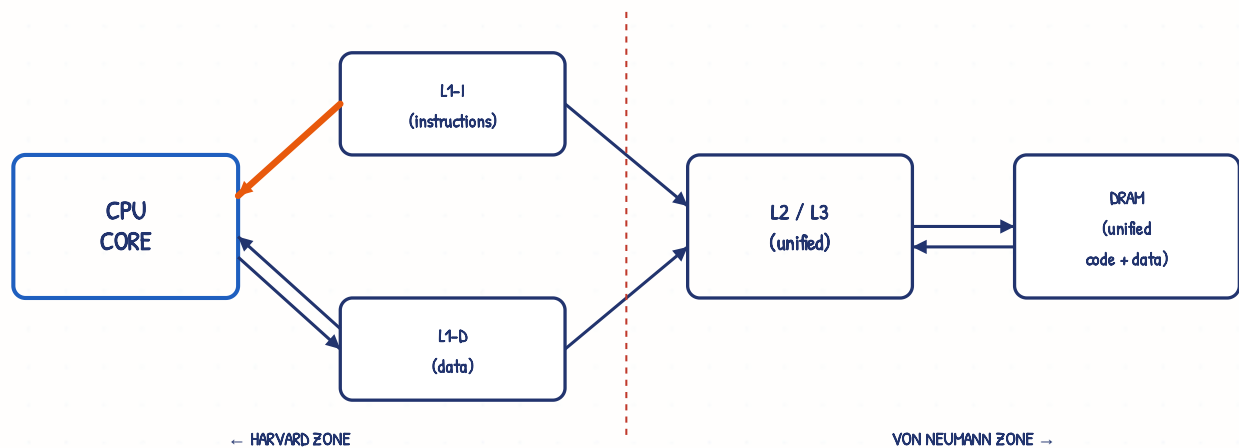
Instruction fetch (I-bus) and data access (D-bus) happen in the **SAME** cycle — no contention.

Harvard block diagram

How to draw: Instruction memory top-left, data memory top-right, CPU in the middle below; a one-way I-bus from instruction memory to CPU and a two-way D-bus between CPU and data memory.

9.5 Modified Harvard

A **modified Harvard architecture** has split instruction and data caches (L1-I, L1-D) inside the chip but a single unified main memory outside — Harvard speed inside, Von Neumann flexibility outside. Used by almost all modern CPUs (x86-64, ARM Cortex-M4).



Split at L1 for parallel fetch; unified from L2 down for simplicity. Modified Harvard in one picture.

Split L1-I / L1-D over unified L2/L3 and DRAM

How to draw: CPU core box with two small caches L1-I and L1-D beside it; both feed one unified L2/L3 box, then DRAM; mark the Harvard zone (L1) and the Von Neumann zone (L2 down).

Yaad rakho: VN = one bus; Harvard = two; modern = split L1 only.

Unit 10 · CISC vs RISC

10.1 Recap and setup

Units 9 → 10: after deciding *how* the CPU connects to memory, the next question is *what instructions* it understands — the ISA.

The **Instruction Set Architecture (ISA)** is the set of instructions, registers and memory model visible to the programmer — the contract between software (which compiles to it) and hardware (which executes it).

10.2 RISC vs CISC

CISC (Complex Instruction Set Computer) provides many powerful, variable-length instructions that can operate directly on memory. **RISC (Reduced Instruction Set Computer)** provides a small set of simple, fixed-length instructions in which only load and store access memory.

CISC	RISC
hundreds of instructions	few simple instructions
variable length (x86: 1-15 bytes)	fixed 32 bits
memory operands allowed	load/store only
complex decoder, microcode	simple decoder
fewer instructions per program	more instructions per program
complexity in hardware	complexity in compiler
x86, VAX, 68000	MIPS, ARM, RISC-V, SPARC

Yaad rakho: RISC hallmarks = fixed length + load/store.

10.3 Historical context

The **ISA family tree** shows how instruction sets evolved along two lineages: the **CISC** line (VAX, 68000, x86 → x86-64) and the **RISC** line (IBM 801 → MIPS, SPARC, ARM, RISC-V).

- CISC: VAX (1977), 68000 (1979), **x86 (1978) → x86-64 (2003)**.
- RISC: IBM 801 → **MIPS (Stanford, Hennessy, 1984)**, SPARC, **ARM (1985)**, **RISC-V (Berkeley, 2010)**.
- ARM won mobile (low power, licensing); x86 kept the desktop (compatibility).

10.4 Compiler implications & code density

Code density is how much work a program does per byte of instructions; CISC has higher code density, RISC is easier to pipeline.

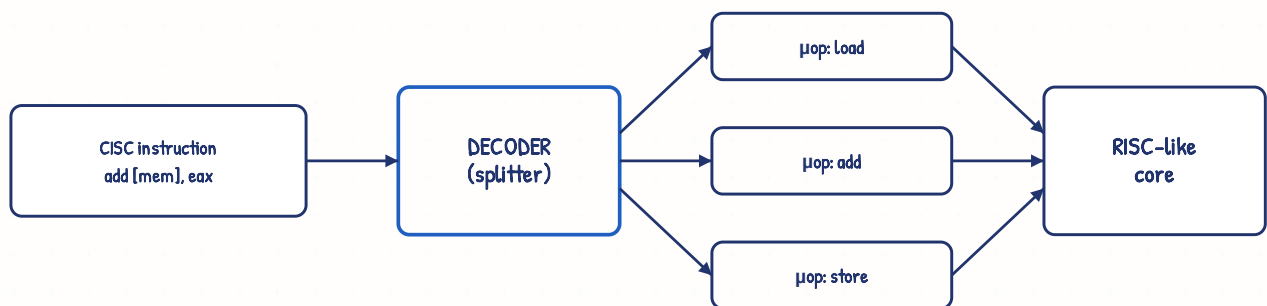
- RISC pipelines well: fixed length, uniform decode, one simple step each, only lw/sw touch memory.
- ARM Thumb and RISC-V “C” (16-bit) improve RISC code density.

$$\text{CPU time} = \text{Instruction count} \times \text{CPI} \times \text{clock period (iron law)}$$

10.5 Micro-ops in x86

Micro-operations (μops) are simple RISC-like internal operations into which a modern x86 decoder splits each complex CISC instruction before executing them on a RISC-style pipelined core.

Hinglish: x86 bahar se CISC hai (purane software chalane ke liye), par andar se RISC jaisa. Decoder ek bada instruction, jaise `add [mem], eax`, ko teen chhote kaam mein todta hai — load, add, store — aur phir unhe RISC ki tarah pipeline mein chalata hai. Isliye kehte hain “CISC outside, RISC inside”.



Front end: complex x86 instruction → several simple μops. Back end: pipelines, renames and reorders μops like a RISC machine.

x86 micro-op decode: CISC skin, RISC core

How to draw: One x86 instruction box → a decoder box → three small μop boxes (load, add, store) → a RISC-like execution core box.

10.6 Trends – RISC-V & AI ISAs

A **domain-specific ISA** is an instruction set designed for one class of workload (e.g. graphics or neural networks), trading generality for much higher efficiency.

- **RISC-V:** open, royalty-free, modular (base + extensions), Berkeley 2010.
- **Domain-specific ISAs:** NVIDIA PTX (GPUs), Google TPU (matrix multiply).

Yaad rakho: “RISC runs fewer instructions” is FALSE — it runs more, but each is simpler.

Unit 11 · MIPS Registers & Instruction Formats

11.1 Why MIPS

MIPS (Microprocessor without Interlocked Pipeline Stages) is a classic RISC ISA from Stanford (1984) with 32 registers and fixed 32-bit instructions in three formats, widely used for teaching.

Clean fixed 32-bit design with only 3 formats, MARS simulator, Patterson & Hennessy textbook, skills transfer to RISC-V/ARM. Used in PlayStation 1, N64, PS2.

11.2 MIPS register set & conventions

MIPS has **32 general-purpose 32-bit registers**; software conventions give each a role. **Caller-saved** registers (\$t) may be changed by a called function; **callee-saved** registers (\$s) must be preserved by it.

Name	No.	Use
\$zero	0	always 0
\$at	1	assembler temp
\$v0-\$v1	2-3	return values
\$a0-\$a3	4-7	arguments
\$t0-\$t7	8-15	temporaries (caller-saved)
\$s0-\$s7	16-23	saved (callee-saved)
\$t8-\$t9	24-25	temporaries
\$k0-\$k1	26-27	OS kernel
\$gp \$sp \$fp \$ra	28 29 30 31	global, stack, frame, return address

Trick: **t** = temporary, don't trust it; **s** = saved, it survives. PC, HI, LO are separate.

11.3–11.5 Instruction Formats (R, I, J)

An **instruction format** is the layout of fields within the 32-bit instruction word. MIPS has three: **R-format** for register-to-register operations, **I-format** for immediates, loads, stores and branches, and **J-format** for jumps.

Hinglish: Har instruction 32 bits ka hota hai, sirf fields ka bantwara alag. R-type mein opcode hamesha 0 hota hai aur operation *funct* batata hai; rd destination hai, rs/rt source. Dhyan do: assembly mein rd pehle likhte hain (add rd, rs, rt) par binary mein order rs, rt, rd hota hai. I-type mein 16-bit constant hota hai; lw aur addi mein rt destination hota hai. J-type mein 26-bit address hota hai jo 4 se multiply hoke (shift left 2)

jump address banta hai. Encode karna: registers ko numbers mein badlo → fields ko binary mein likho → 4-4 bits ke group → hex.

R: opcode 6 | rs 5 | rt 5 | rd 5 | shamt 5 | funct 6 (bits 31-26 | 25-21 | 20-16 | 15-11 | 10-6 | 5-0)

I: opcode 6 | rs 5 | rt 5 | immediate 16 · **J:** opcode 6 | address 26

Effective address = **rs + signext(imm)** · Branch target = **PC + 4 + imm × 4** · Jump = **PC+4[31:28] || addr || 00**

funct: add 0x20, sub 0x22, and 0x24, or 0x25, slt 0x2A · opcode: lw 0x23, sw 0x2B, beq 4, addi 8, j 2, jal 3

Example: add \$t1, \$t2, \$t3 → 000000 01010 01011 01001 00000 100000 → **0x014B4820**

lw \$t1, 16(\$sp) → 100011 11101 01001 00000000000010000 → **0x8FA90010**

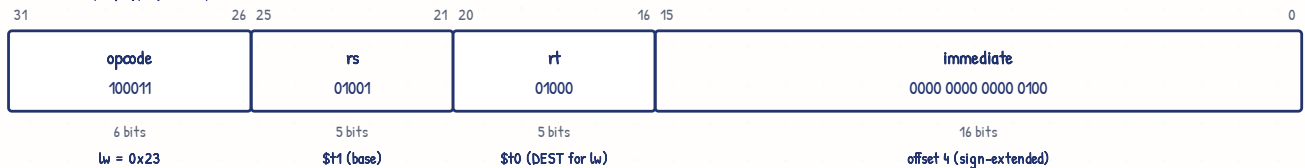
R-format: add \$t1, \$t2, \$t3 = 0x014B4820



R-format fields (add \$t1, \$t2, \$t3)

How to draw: A 32-bit bar split 6|5|5|5|5|6 with bit numbers 31-26, 25-21, 20-16, 15-11, 10-6, 5-0; write the field names and the binary values inside.

I-format: lw \$t0, 4(\$t1) = 0x8D280004



I-format fields (lw \$t0, 4(\$t1))

How to draw: A 32-bit bar split 6|5|5|16; label opcode, rs (base), rt (destination for loads), immediate (offset); write the binary.

Yaad rakho: R = 6-5-5-5-5-6, I = 6-5-5-16, J = 6-26. rd = destination (R), rt = destination (lw, addi).

11.6 RISC-V modular ISA

RISC-V is an open, royalty-free RISC ISA built as a small base (RV32I, ~40 instructions, x0-x31) plus optional standard extensions.

Formats R, I, S, B, U, J · Extensions **M** multiply, **A** atomic, **F** float, **D** double, **C** compressed, **V** vector (e.g. RV32IMAFD)

11.E1 Assembly toolkit (Lab 11)

An **assembler directive** (.data, .text, .word, .ascii) instructs the assembler how to lay out the program; a **pseudo-instruction** (li, la, move, blt) is a convenience the assembler expands into real instructions; a **syscall** asks the operating system for a service (print, read, exit) chosen by the code in \$v0.

- syscall: 1 print int, 4 print string, 5 read int, 10 exit, 11 print char (put code in \$v0).
- div → LO = quotient, HI = remainder (mflo, mfhi). mult → HI:LO.
- **la** loads an address, **lw** loads a value. lw/sw addresses must be multiples of 4.
- Pseudo-ops: li, la, move, blt, bge, nop. srl fills 0s, sra copies the sign bit.
- Endianness of 0x12345678: big-endian first byte 12, little-endian first byte 78.

Unit 12 · C to MIPS Translation

12.1 Translation pattern

Translating C to MIPS means mapping variables to registers, conditions to compare-and-branch instructions, and control structures (if, loops, functions) to labelled blocks connected by branches and jumps.

- Variables → registers. Conditions → slt + beq/bne. Structure → labels.
- **Reverse the condition**: branch AWAY from the body when the test fails.
- Array: **address of A[i] = base + 4 × i**.

12.2 if / else

A **conditional branch** (beq, bne) changes the program counter to a label only if its condition is true; otherwise execution continues with the next instruction. An **if/else** is translated with one conditional branch and one unconditional jump.

Hinglish: MIPS mein sirf beq (barabar) aur bne (barabar nahi) hote hain. "Agar a == b toh body chalao" ko aise likhte hain: "agar a != b toh body ko skip karo" (bne). Isko condition ulta karna kehte hain. Chhota-bada check karna ho toh pehle slt se flag banao (a < b ho toh 1) phir flag ko \$zero se compare karo. a > b ke liye operands ulte kar do: slt t, b, a. if/else mein if-block ke end mein "j Done" zaroori hai warna else-block bhi chal jayega.

```
bne $t0, $t1, Else    # if (a == b)
li  $t2, 1            # c = 1
j   Done              # skip the else part
Else: li $t2, 0        # else c = 0
Done:
```

Yaad rakho: a > b → slt t, b, a. Always end the if-block with j Done.

12.3 Loops

A **loop** in assembly is a labelled block that tests a condition, executes its body, updates the loop variable and jumps back to the label until the condition fails.

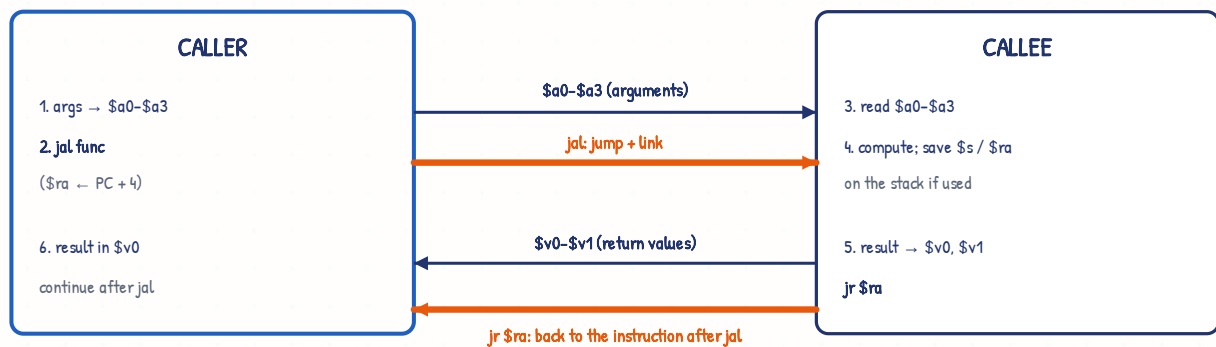
```
Loop: bge $t0, $t1, End # while (i < n): leave when i >= n
add  $s0, $s0, $t0     # sum += i
addi $t0, $t0, 1       # i++
j    Loop
End:
```

Yaad rakho: init → test (branch out) → body → increment → jump back. (The class slide's blt \$t1,\$t0 version runs one extra time.)

12.4 Calling convention

The **calling convention** is the agreed set of rules for function calls: arguments in $\$a0-\$a3$, return values in $\$v0-\$v1$, the return address in $\$ra$ set by `jal`, and return with `jr $ra`.

jal Label: $\$ra \leftarrow PC + 4$, then jump · **jr \$ra:** return



More than four arguments → extra ones go on the stack. $\$t$ registers may be clobbered by the callee; $\$s$ must be preserved.

Caller / callee register flow

How to draw: Two boxes (caller, callee); arrows right labelled $\$a0-\$a3$ and `jal`; arrows left labelled $\$v0$ and `jr $ra`; number the six steps.

12.5 Stack frames — leaf vs non-leaf

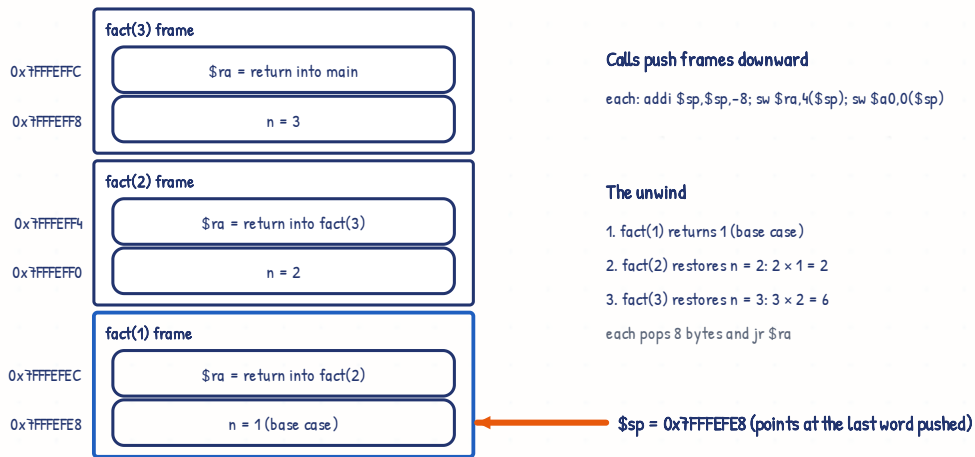
A **stack frame** is the block of stack memory a function allocates to save registers and local data. A **leaf function** calls no other function; a **non-leaf function** calls others and must save $\$ra$ (and any needed $\$a/\s registers) on the stack.

Hinglish: Stack neeche ki taraf badhta hai — jagah banane ke liye $\$sp$ ghatao (`addi $sp, $sp, -8`), wapas dene ke liye badhao. Jo function khud kisi aur ko call karta hai, uska $\$ra$ jal se overwrite ho jaata hai, isliye pehle $\$ra$ stack pe save karo. Recursive factorial mein har call apna $\$ra$ aur n save karta hai; `fact(3)` mein 3 frames bante hain, phir ulte order mein khulte hain: $1 \rightarrow 2 \times 1 = 2 \rightarrow 3 \times 2 = 6$.

Push: `addi $sp, $sp, -8; sw $ra, 4($sp); sw $a0, 0($sp)` · Pop: `lw back, addi $sp, $sp, 8`

$\text{fact}(n)$ peak stack = $n \text{ frames} \times 8 \text{ bytes}$ (`fact(3)` = 24 bytes)

Stack at the deepest point of fact(3)



Calls push frames downward

each: `addi $sp,$sp,-8; sw $ra,4($sp); sw $a0,0($sp)`

The unwind

1. fact(1) returns 1 (base case)
2. fact(2) restores n = 2: $2 \times 1 = 2$
3. fact(3) restores n = 3: $3 \times 2 = 6$

each pops 8 bytes and `jr $ra`

3 calls → 3 frames (24 bytes) at peak, then unwound one by one. Addresses assume `$sp = 0x7FFFF000` before the first call.

fact(3): frames at the deepest point and the unwind

How to draw: Three stacked frames (fact(3), fact(2), fact(1)), each holding saved \$ra and n; \$sp arrow at the bottom frame; list the three unwind steps.

Yaad rakho: stack grows DOWN. Non-leaf must save \$ra. Keep the stack balanced.

Unit 13 · Single-Cycle MIPS Datapath

13.1 ISA → hardware

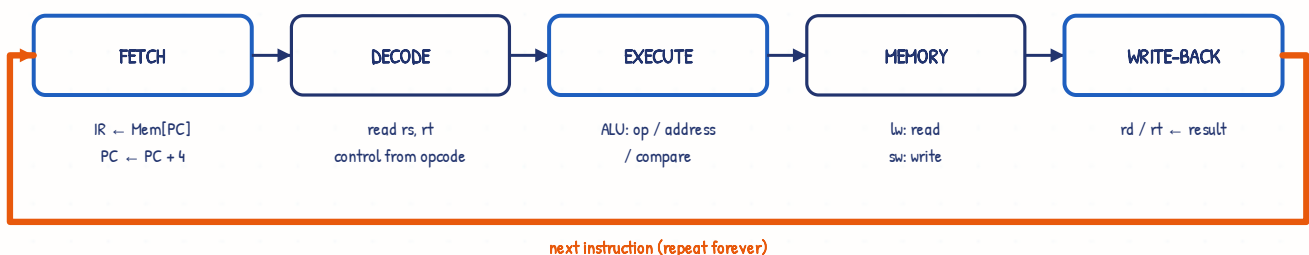
The **datapath** is the hardware that stores and processes data (PC, memories, register file, ALU, MUXes); the **control unit** generates the signals that tell the datapath what to do for each instruction. In a **single-cycle** design each instruction completes in one clock cycle.

- CPI = 1, but clock period = slowest instruction (lw).
- Separate instruction and data memories, because each unit is used once per cycle.

13.2 Fetch-decode-execute cycle

The **instruction cycle** is the repeated sequence a CPU follows for every instruction: **fetch** it from memory, **decode** it and read registers, **execute** it in the ALU, access **memory** if needed, and **write back** the result.

Fetch: $IR \leftarrow \text{Mem}[PC]$, $PC \leftarrow PC + 4$ · add skips Memory · sw skips Write-back · beq may change the PC



Single-cycle: all five steps happen within ONE long clock cycle. Pipelined (Unit 14): each step is its own stage.

Fetch-decode-execute-memory-write-back loop

How to draw: Five boxes in a row with arrows, and one arrow looping from write-back back to fetch; write what each step does under it.

13.3 Building the datapath

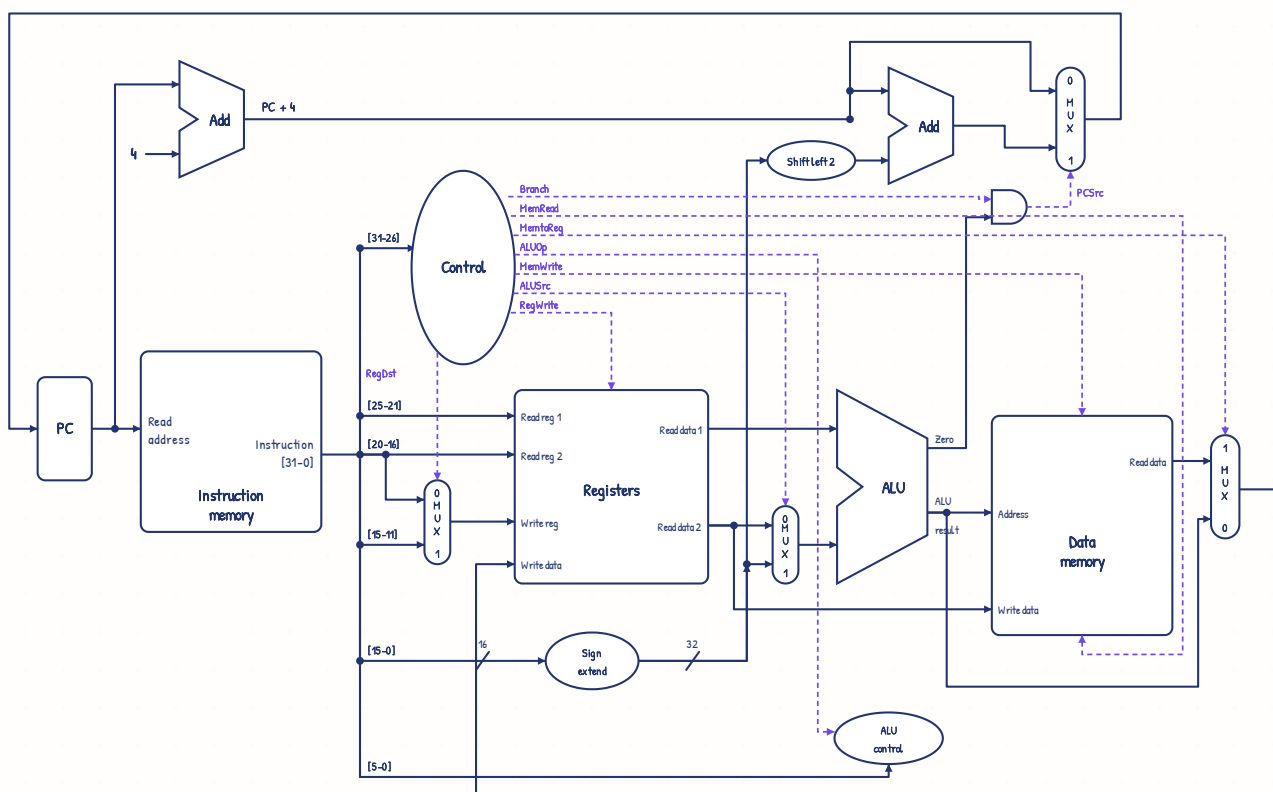
The **single-cycle datapath** connects the PC, instruction memory, register file, sign-extender, ALU, data memory and multiplexers so that every instruction type (R-type, lw, sw, beq) can use the parts it needs; the **main control unit** decodes the opcode into control signals and the **ALU control** uses ALUOp and funct to select the ALU operation.

Hinglish: Instruction ke bits alag-alag jagah jaate hain: [31-26] opcode → Control; [25-21] rs → Read reg 1; [20-16] rt → Read reg 2; [15-11] rd → RegDst MUX; [15-0] → sign extend. Chaar MUX yaad karo: RegDst (rt ya rd — kis register mein likhna), ALUSrc (register ya immediate — ALU ka doosra input), MemtoReg (ALU

result ya memory data – register mein kya likhna), PCSrc (PC + 4 ya branch target). Branch tabhi hota hai jab Branch = 1 AND ALU ka Zero = 1.

Instr	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp
R-type	1	0	0	1	0	0	0	10
lw	0	1	1	1	1	0	0	00
sw	X	1	X	0	0	1	0	00
beq	X	0	X	0	0	0	1	01
addi	0	1	0	1	0	0	0	00

$PCSrc = \text{Branch AND Zero} \cdot \text{ALUOp } 00 \text{ add, } 01 \text{ sub, } 10 \text{ funct} \cdot \text{ALU control add } 0010, \text{ sub } 0110$



Single-cycle datapath with control (Patterson & Hennessy style) – dashed = control signals

Single-cycle MIPS datapath with control (as printed on the class worksheets)

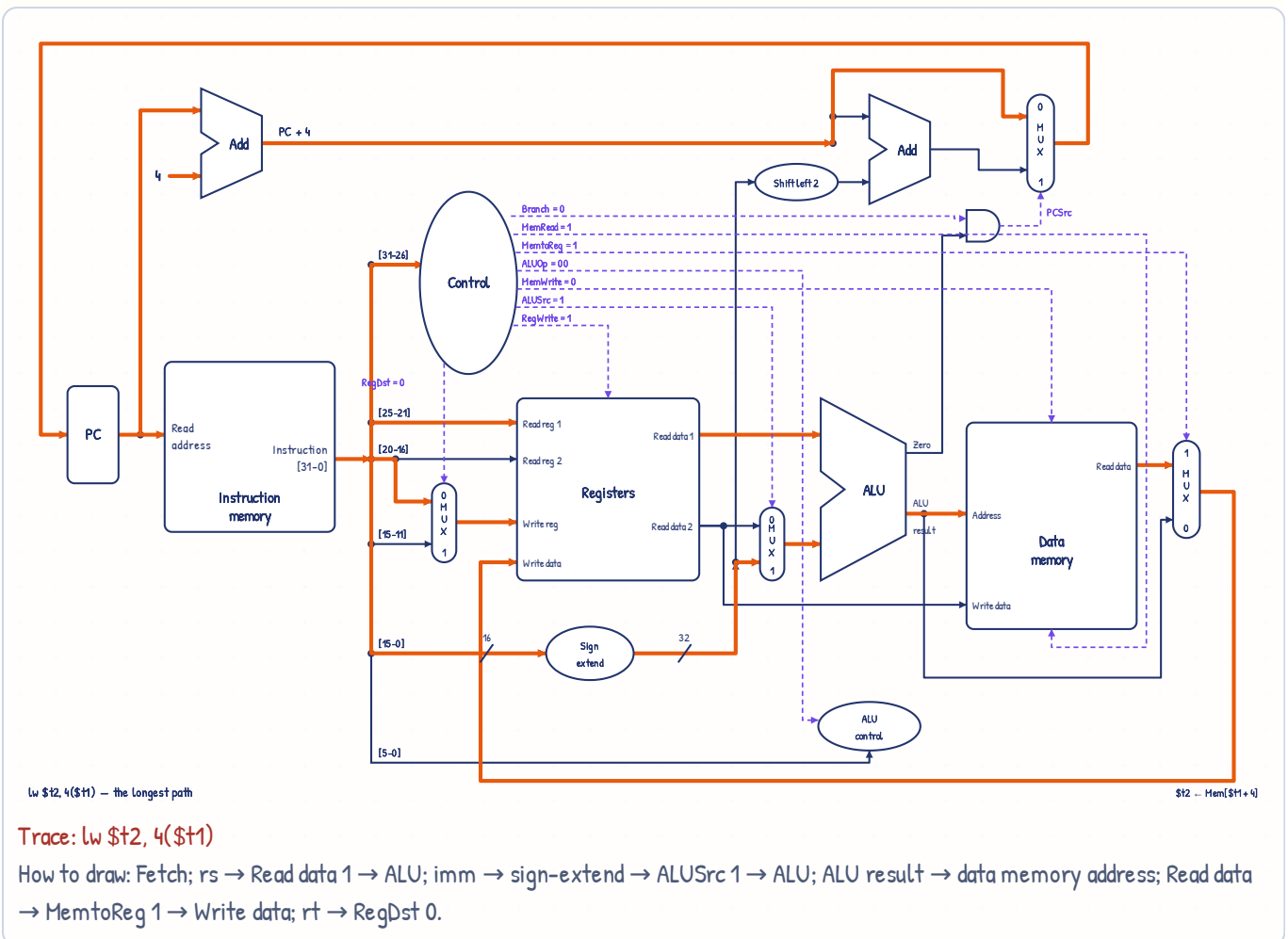
How to draw: Five big blocks left to right (PC, instruction memory, registers, ALU, data memory); add the PC + 4 adder and loop; insert RegDst, ALUSrc, MemtoReg and PCSrc MUXes; control ellipse above the register file with dashed lines to each MUX, memory and RegWrite; finally sign-extend, shift-left-2, branch adder and AND gate.

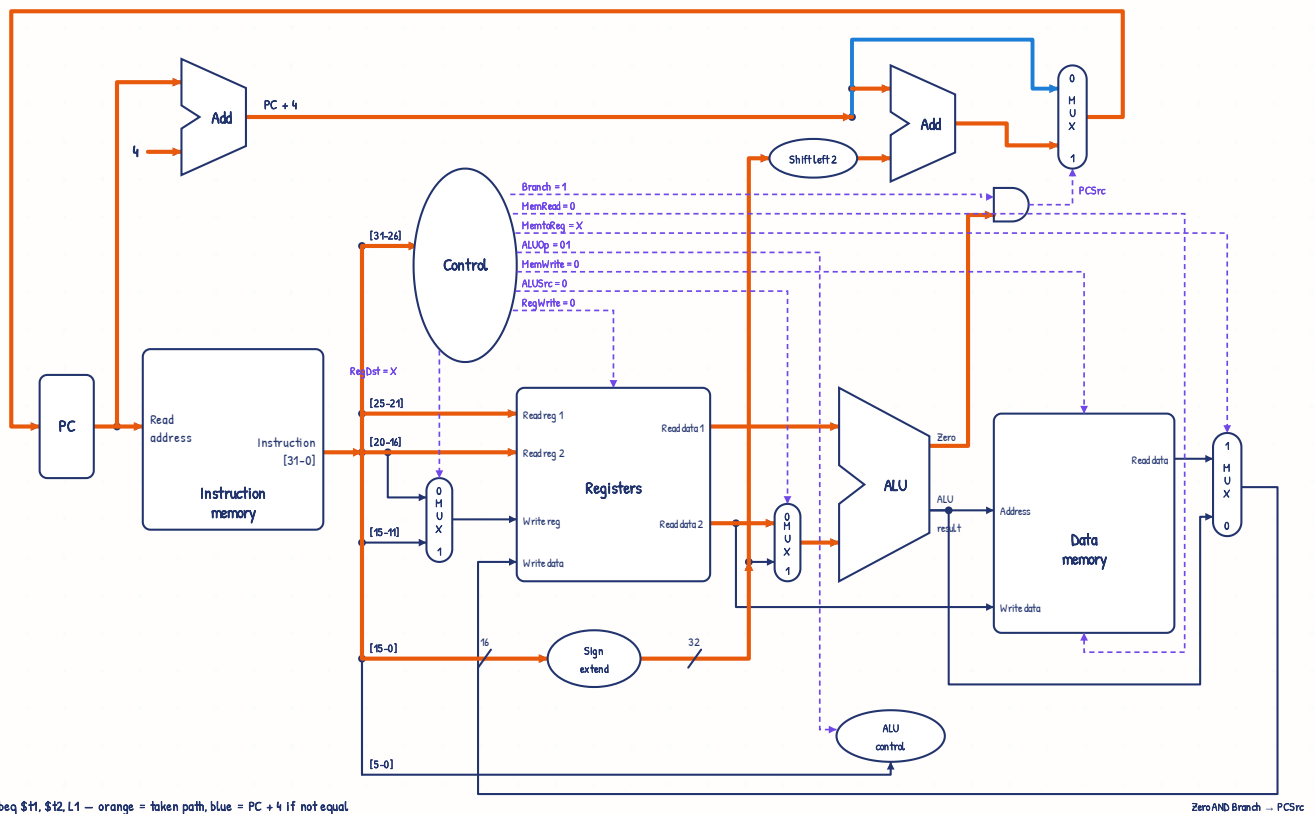
Trick: X (don't care) only on RegDst and MemtoReg, and only when RegWrite = 0.

13.4 Tracing instructions

Tracing an instruction means marking the path its data takes through the datapath and writing the value of every control signal for it.

- Step 1 fetch path → step 2 follow the fields → step 3 mark the MUX inputs used → step 4 fill the control row.
- **Never Leave ALUOp blank** (lw/sw 00, beq 01, R-type 10).





Trace: beq \$t1, \$t2, L1

How to draw: Fetch; rs and rt into the ALU (ALUSrc 0) to subtract; offset → sign-extend → shift-left-2 → branch adder with PC + 4; Zero AND Branch selects the PCSrc MUX input.

13.5 Critical path & clock period

The **critical path** is the longest delay path through the circuit between two clock edges; it sets the minimum clock period. In the single-cycle datapath it is the path of **lw**.

$lw = IMem + Reg\ read + ALU + DMem + Reg\ write = 200 + 100 + 200 + 200 + 100 = 800\ ps\ (P\&H)$
 $sw\ 700 \cdot R\text{-type}\ 600 \cdot beq\ 500 \cdot j\ 200 \rightarrow clock = 800\ ps = 1.25\ GHz$

Yaad rakho: lw is the slowest → it decides the single-cycle clock.

Unit 14 · Pipelining (shorter – no class slides)

14.1–14.2 The idea

Pipelining is a technique in which the execution of an instruction is divided into stages and several instructions are overlapped, each in a different stage, so that one instruction can complete every clock cycle.

Hinglish: Laundry socho – ek load dhul raha ho toh doosra dryer mein, teesra fold ho raha. Har load ka time utna hi hai (latency same), par har 30 minute mein ek load ready (throughput badha). CPU mein bhi ek instruction fetch ho rahi hoti hai, doosri decode, teesri execute.

14.3–14.6 Five stages, diagrams, speedup, hazards

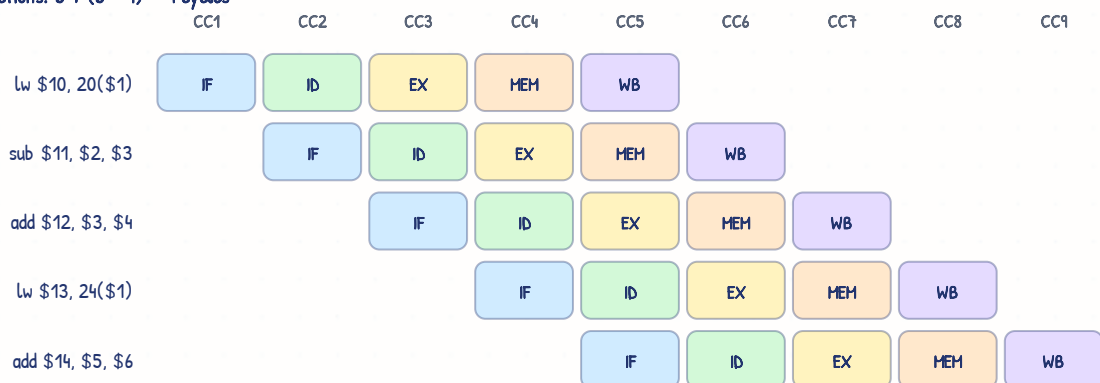
A **pipeline register** is a register placed between two pipeline stages that holds everything one stage produced (data, register numbers, control signals) for the next stage. **Throughput** is instructions completed per unit time; **latency** is the time for one instruction to pass through all stages.

- Stages IF, ID, EX, MEM, WB; pipeline registers IF/ID, ID/EX, EX/MEM, MEM/WB.
- Pipeline diagram: rows = instructions, columns = clock cycles.
- Hazards: structural, data, control (Unit 15).

Cycles = $k + (n - 1)$ · Speedup = $n \cdot k \div (k + n - 1) \rightarrow k$ · Clock = **slowest stage** (+ register overhead)

Example: stages 200/100/200/200/100 ps → clock 200 ps → speedup $800/200 = 4$

Five independent instructions: $5 + (5 - 1) = 9$ cycles



Multi-cycle pipeline diagram (no hazards)

How to draw: Grid with instructions down the left and CC1–CC9 across the top; write IF ID EX MEM WB along a diagonal staircase, each row shifted one column right.

Yaad rakho: pipelining improves throughput, not latency.

Unit 15 · Pipeline Hazards (shorter – no class slides)

15.1–15.2 Hazard types

A **hazard** is a situation in which the next instruction cannot execute in its planned clock cycle.

Structural: two instructions need the same hardware. **Data**: an instruction needs a result not yet produced. **Control**: the next instruction depends on an unresolved branch.

- RAW (read after write) is the only data hazard in the in-order 5-stage pipeline; WAR and WAW occur only in out-of-order machines.

15.3–15.6 Stalls, forwarding, load-use, scheduling

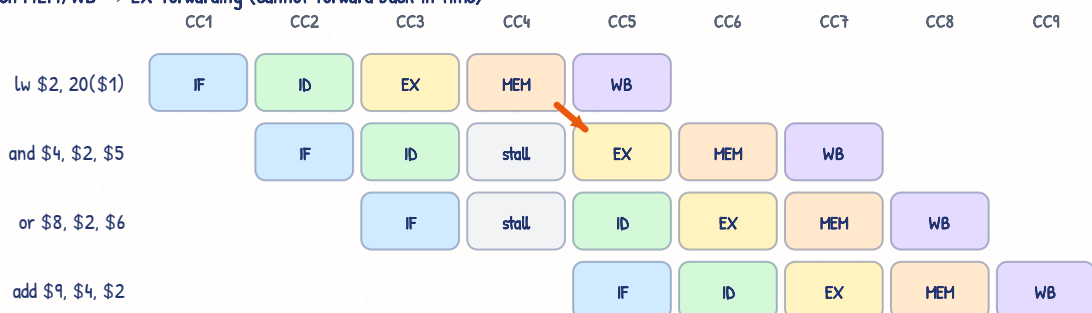
Stall (bubble): delaying an instruction until its operand is ready. **Forwarding (bypassing)**: sending a result directly from a pipeline register to the ALU input of a later instruction instead of waiting for write-back.

Hinglish: Bina forwarding, agla instruction tab tak ruka rahega jab tak result register mein likha na jaaye – 2 cycle ka wait. Forwarding mein result seedha EX/MEM ya MEM/WB register se ALU tak bhej dete hain, toh wait nahi. Sirf load ke turant baad use karne pe (load-use) 1 cycle rukna padta hai, kyunki memory se data MEM stage ke end mein aata hai. Compiler instructions ka order badal ke ye stall bhi hata sakta hai.

No forwarding: dependent next instruction → **2 stalls** · Forwarding: ALU → ALU **0 stalls** · Load-use: **1 stall**

Paths: EX/MEM → EX, MEM/WB → EX · Scheduling example: 13 cycles → 11 cycles

Load-use: one bubble, then MEM/WB → EX forwarding (cannot forward back in time)



Load-use: 1 stall + forwarding

How to draw: lw normal; and fetched next with one stall box before its EX; arrow from the end of lw's MEM to the start of and's EX; later instructions shift right by one.

Yaad rakho: 2 / 0 / 1 – stalls without forwarding / with forwarding / load-use.

Last-hour cheat sheet

Gates

XOR = $A'B + AB'$ (differ) · XNOR = $A'B' + AB$ (equal) ·
NAND/NOR universal

Laws

$A + AB = A \cdot A(A + B) = A \cdot A + A'B = A + B \cdot AB + AB' = A$
De Morgan: $(AB)' = A' + B'$, $(A + B)' = A'B'$

Canonical / K-map

SOP: $1 \rightarrow X$, $0 \rightarrow X'$ · POS: $0 \rightarrow X$, $1 \rightarrow X'$
Gray 00 01 11 10 · group 2^k removes k variables · corners wrap

Universal gate counts

NAND: NOT 1, AND 2, OR 3, XOR 4 · NOR: NOT 1, OR 2, AND 3
SOP $\rightarrow$ NAND-NAND · POS $\rightarrow$ NOR-NOR

MUX / decoder

2:1Y = $S'I_0 + SI_1$ · n selects $\rightarrow 2^n$ inputs · decoder $D_k = m_k$

Adders

HA: $S = A \oplus B$, $C = AB$ · FA: $S = A \oplus B \oplus C_{in}$, $C_{out} = AB + C_{in}(A \oplus B)$
CLA: $G = AB$, $P = A \oplus B$, $C(i+1) = G_i + P_i C_i$ · ALU: add 0010, sub 0110

Numbers

2's range $-2^{n-1} \dots 2^{n-1} - 1$ · $-x = \text{invert} + 1$ · MSB weight -2^{n-1}
 $A - B = A + B' + 1$ · $V = C_3 \oplus C_4$ · $CF = C_4$ · sign-extend = copy MSB

Latches / FFs

NOR SR: 11 forbidden · NAND SR: 00 forbidden · $Q^+ = S + R'Q$
D latch: transparent when $EN = 1$ · D FF: $Q^+ = D$ at the edge
JK: $Q^+ = JQ' + K'Q$ · T: $Q^+ = T \oplus Q$ · $T = Q \oplus Q^+$

Timing

$T_c \geq t_{pcq} + t_{pd} + t_{setup}$ · $f = 1/T_c$ · hold: $t_{ccq} + t_{cd} \geq t_{hold}$
ring oscillator $T = 2 \cdot N \cdot t_{pd}$ · $1/ns = 1\text{ GHz}$

Counters / registers

FFs = $\lceil \log_2 N \rceil$ · ripple delay $n \cdot t_{pd}$ · bit $k = f/2^{(k+1)}$
 $T_0 = 1$, $T_1 = Q_0$, $T_2 = Q_0 Q_1$ · load enable $Q^+ = LD + L'Q$
ring n , Johnson $2n$, LFSR $2^n - 1$ · SIPO = UART RX, PISO = UART TX

Register file / FSM

32×32 , 2 read + 1 write · address bits = $\log_2(\text{regs})$
read = MUX (combinational), write = decoder + WE (clocked)
Moore: output in circle; Mealy: on arrow · 1011: Mealy 4 states, Moore 5

Memory

SRAM 6T, no refresh, cache · DRAM 1T1C, refresh 64 ms, destructive read
refresh interval = $64\text{ ms} \div \text{rows}$ ($8192 \rightarrow 7.8\text{ }\mu\text{s}$) · AMAT = hit + miss rate $\times$ penalty

Architecture

VN: one bus (bottleneck, Backus 1977) · Harvard: two buses · modern: split L1
bandwidth = $\text{bits/cycle} \times f \div 8$ · RISC = fixed length + load/store · CPU time = $IC \times CPI \times T$

MIPS

R 6-5-5-5-5-6 · I 6-5-5-16 · J 6-26 · \$zero 0, \$v 2-3, \$a 4-7, \$t 8-15, \$s 16-23, \$sp 29, \$ra 31
 $EA = rs + \text{signext}(\text{imm})$ · branch = $PC + 4 + \text{imm} \times 4$ · jump = $PC + 4[31:28] \parallel \text{addr} \parallel 00$
add 0x20, sub 0x22, slt 0x2A, lw 0x23, sw 0x2B, beq 4, addi 8, j 2
div: LO = quotient, HI = remainder · $A[i] = \text{base} + 4i$ · jal: \$ra = $PC + 4$

Datapath

R: RegDst 1, ALUSrc 0, MemtoReg 0, RegWrite 1, ALUOp 10
lw: 0, 1, 1, 1, MemRead 1, ALUOp 00 · sw: X, 1, X, 0, MemWrite 1, 00
beq: X, 0, X, 0, Branch 1, ALUOp 01 · PCSrc = Branch · Zero · lw = critical path

Pipeline

IF ID EX MEM WB · cycles = $k + n - 1$ · speedup $\rightarrow k \cdot \text{clock} = \text{slowest stage}$
stalls: 2 (no forwarding) / 0 (forwarding) / 1 (load-use)